EDUSPHERE MANAGEMENT SYSTEM (EMS)

By

ZUBAIR AHMAD

MURAD AHMED

A Thesis submitted to The University of Swabi, Khyber Pakhtunkhwa, in partial fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

UNIVERSITY of SWABI

Supervised By:
Dr. Ameer Shakayb Arsalan

DEPARTMENT OF COMPUTER SCIENCE
THE UNIVERSITY OF SWABI
KHYBER PAKHTUNKHWA, PAKISTAN

2022-2026

EDUSPHERE MANAGEMENT SYSTEM (EMS)

By

ZUBAIR AHMAD

MURAD AHMED

A Thesis submitted to The University of Swabi, Khyber Pakhtunkhwa, in partial fulfillment of the requirements for the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Approved by:

Supervisor	Dr. Ameer Shakayb Arsalan
Chairman	Prof. Dr. M Irfan Uddin
External Examiner	
Dean Faculty of Science	Dr. Mukarram Shah

DEPARTMENT OF COMPUTER SCIENCE
THE UNIVERSITY OF SWABI, KHYBER PAKHTUNKHWA, PAKISTAN
SESSION 2022-2026

TABLE OF CONTENT

LIST OF TABLES

LIST OF LIST OF FIGURES

ACKNOWLEDGMENTS

All glory be to Allah Almighty whose countless blessings granted us the privilege of successfully executing this research task.

Firstly, we extend our heartfelt appreciation to our parents who gave their unconditional support in making this whole journey successful. It is their sacrifices that made our education possible, and this thesis is offered to them.

Our sincere gratitude goes to our supervisor, Dr. Ameer Shakayb Arsalan who provided us with all sorts of guidance in carrying out this research task. We are thankful to him for solving our problems technically, which was one of the challenging tasks in carrying out this project. Apart from this, he taught us how to think independently and logically.

We are indebted to the teaching faculty of the Department of Computer Science, University of Swabi, for providing such an environment where we could learn and perform better. Their guidance in lab assignments and course material helped us build a robust base for this project.

Our appreciation also goes to our friends and other students who were there for us during the tough moments. Their guidance and cooperation proved to be helpful in getting this task accomplished.

We are thankful to our partner in this project, Mr. Murad Ahmed, who handled the frontend of the project with dedication and zeal.

Zubair Ahmad
Murad Ahmed

ABSTRACT

ZUBAIR AHMAD, MURAD AHMED
Department of Computer Science
University of Swabi
Session 2022-2026

Supervisor:Dr. Ameer Shakayb Arsalan

A institutions needs a good system to manage its students, teachers, and courses. Without such a system, teachers send assigment ,result through email or WhatsApp. Students submit work on paper. This creates confusion and delays.

We built EduSphere to fix these problems. EduSphere is a web based Learning Management System. Students can log in and see their courses. They submit assignments online. Teachers create courses and upload materials. They give grades and write feedback. Parents make their own accounts. They link to their child and see grades and attendance. Administrators control everything from one place.

We used python (django) for the backend. sqllite stores all data. HTML, CSS, and JavaScript build the frontend. The system runs on any web browser. No software installation is needed.

We tested EduSphere with real users. Teachers said it saves time. Students found it easy to use. The system worked well. It handles many users at once without slowing down.

This thesis explains how we built EduSphere. We show the design, the code, and the test results. Other institutions can use this work to build their own systems.

Keywords: EDUSPHERE Management System, EduSphere, Web Application, python,Django, sql, Education Technology,Gitub and PytonAnyWherefor deployment

CHAPTER 1

Introduction

1.1 Project Background And Overview

Colleges and universities have changed a lot. Not just the way they teach. Also the way they manage things. Ten or fifteen years back, everything was on paper. Teachers wrote attendance in a register. Students submitted homework on loose sheets. The office had big steel cabinets full of files. It worked okay when there were fewer students. But now? Classrooms are full. Hundreds of students enroll every year. Records pile up. Files get lost. Papers fall out from files.

Every day, universities collect so much information. Student names. Roll numbers. Attendance marks. Test scores. Late fees. Leave applications. It never stops. Doing all of this by hand is not just slow. It is a headache. One small mistake creates big problem. A teacher marks a student absent by accident. That student loses attendance. The parent gets angry. The teacher has to find the original register. Hours wasted. All because of one wrong tick mark.

Eduspahre can fix this. That is why we started working on EduSphere. It is a Learning Management System. But not just any system. We built it to solve the real problems we saw every day on campus. The idea is simple. Put everything in one place. Courses. Assignments. Grades. Attendance. Everything. When a teacher uploads a file, every student sees it right away. No more "I did not get the message." No more missed assignments. No more excuses. When a parent wants to check progress, they log in. No call to the teacher. No visit to the office. Just open the phone and look. When an administrator needs a report, they click one button. No searching through registers. No calling different departments. Just data. Fast and clean. The system knows who you are. A student logs in and sees their courses. A teacher logs in and sees their classes. An admin sees everything. Same login page. Different views. Smart and simple. We connected three main users. The admin who runs the show. The teacher who does the teaching. The student who does the learning. Everyone gets what they need. No confusion. No delays. No lost files. That is the goal of this project. Build something that actually helps. Something that saves time. Something that makes university life a little easier for everyone.

1.2 Problem Description

Before we built anything, we sat down and looked at how universities really work. Not the fancy brochures. Not what they say on open day. But the real day to day mess.

What we found was simple. Most institutions still live in the past. Some use papers. Some use a little bit of computer and a lot of manual work. No One talks to each

other. Attendance is in one notebook. Grades are in another file. Fees are on a different spreadsheet.

This creates problems.

1.2.1 The Admin Mess

The administration office is the busiest place on campus. Students line up outside. Parents call every hour. Teachers come with requests. And what do the staff have? Papers. Lots of papers.

When a new student joins, someone write everything by hand. Name. Father name. Address. Phone. Roll number. One spell mistake and the record is wrong. Then the student complains. Then someone has to find that one paper among thousands. Then fix it. Then print it again. Then file it again.

Fees are worse. Some students pay on time. Some pay late. Some pay half this month and half next month. The fee person opens multiple Excel files. They color code rows. Red for not paid. Yellow for partial. Green for paid. But the files get messy. A wrong click changes a formula. Numbers become wrong. Parents get angry calls.

The admin has no single screen to see everything. No dashboard. No quick reports. They just have folders and files and tired eyes.

1.2.2 The Teacher's Load

Teachers are hired to teach. But most of their time goes somewhere else.

Attendance takes ten minutes every class. Call each name. Mark present or absent. Count who is missing. Write it down. Then enter it into another sheet at the end of the month. That is hours of work every week. Hours that could be used for teaching.

Assignments are another headache. One student prints their work and hands it physically. Another student emails their file. A third brings a USB drive. The teacher collects from three different places. Papers get mixed up. Files get lost in email spam. USBs get viruses.

Then grading starts. The teacher opens each file. Reads it. Writes a number on paper. Adds everything manually. One addition mistake and the final grade is wrong. A student complains. The teacher checks again. Finds the mistake. Fixes it. Apologizes.

There is no single place where teachers can see all their subjects. No quick way to upload materials. No easy system to give feedback. Everything takes ten steps when it should take one.

1.2.3 The Student's Struggle

Student never know where they stand. Attendance percentage? Ask the teacher. Assignment grade? Ask the teacher. Exam result? Wait for the notice board. Everything is a chase. Every answer takes days.

Assignment briefs come on WhatsApp. The file gets buried under family messages. The student cannot find it. The deadline passes. The teacher says sorry. The student

fails.

Transcripts are even worse. A student needs one paper for a job interview. They go to the office. Stand in line for an hour. Fill out a form. Come back after three days. Stand in line again. Get told "come tomorrow." This should take five minutes online. It takes one week in real life.

Students have no power. They cannot see their own records. They cannot download their own documents. They cannot check their own attendance. Everything goes through someone else.

1.2.4 Summing It Up

Three problems. Admin mess. Teacher load. Student struggle. Each one is bad alone. Together they make university life hard for everyone. We saw these problems every day. We decided to fix them. EduSphere is our answer.

1.3 Project Objectives

We had one big goal. Build an EMS/LMS that actually works. Not something fancy. Something useful. Something that fixes the three problems I just talked about.

But one big goal is not enough. We broke it down into smaller pieces. Here is what we wanted to achieve.

1.3.1 Secure Login For Everyone

First thing first. The system must know who is logging in.

A student should not see teacher menus. A teacher should not see admin menus. Sounds simple. But many systems get this wrong. They show everything to everyone. Buttons that do not work. Options that lead nowhere. Confusing. We built a clean login system. You type your email and password. The system checks your role. Then it shows only what you need.

An admin sees the full dashboard. All controls. All reports.

A teacher sees their classes, their students, their assignments. Nothing else.

A student sees their courses, their grades, their attendance. That is it.

No confusion. No extra buttons. Just what matters.

1.3.2 Admin Dashboard That Works

The administrator has the hardest job. They need to see everything. Control everything. But they should not need a computer science degree to do it.

We built a simple admin panel. Add a new user? Two clicks. Remove a student One click. Change a teacher's schedule?

Fee tracking is built in. The admin sees who paid and who did not. No more color coded Excel sheets. No more manual highlighting.

Discipline cases are also digital. A teacher files a report. The admin sees it. Parents get notified. Everything tracked. Nothing lost.

Timetable creation used to take two days. Now it takes twenty minutes.

1.3.3 Teacher Tools That Save Time

Teachers are busy. They do not have time to learn complicated software. They need things to work fast.

We gave them a simple dashboard. On the left side, all their subjects. Click once to open. Inside each subject, three things. Materials. Assignments. Grades.

To create an assignment? Type the title. Type the due date. Upload the instructions Done.

To take attendance? A list of students appears. Click present or absent. Done. The system saves everything automatically.

To grade assignments? Open each student's work. Type a number. Write feedback. Done. The system calculates the final grade. No manual addition. No mistakes.

What used to take three hours now take 2 minutes.

1.3.4 Student Dashboard That Empowers

Students should not have to beg for information. They should see everything themselves.

We built a student dashboard that shows everything. Daily timetable appears first. Which class. Which room. Which teacher. No confusion.

All course materials are in one place. The teacher uploads once. Every student sees it right away. No more "I did not get the file."

Assignments are listed with due dates. Green means on time. Red means late. Yellow means tomorrow. The student never misses a deadline.

Grades appear as soon as the teacher enters them. No waiting for result day. No chasing teachers. Just open the dashboard and look.

Need a transcript? Click a button. Fill the form online. Wait one day. Download the PDF. No standing in lines. No filling paper forms. No coming back after three days.

The student is in control now.

1.3.5 No More Lost Data

This was our final goal. Keep everything safe. All data lives in one database. Not ten different Excel files. Not paper registers. Not USB drives. One central place. If a computer crashes, nothing is lost. Just login into another system. No panic. No "sorry.

Everyone sees the same up to date information. The teacher enters a grade. The student sees it immediately. The parent sees it immediately. No delays. No "check back next week."

1.3.6 Did We Achieve These Goals?

Yes. Mostly.

The login system works perfectly. Different roles see different things. No bugs. The admin panel does everything we planned. Some small features took longer than expected. But they work now.

Teachers saved time. We test it by our teacher. He said "Why did we not have this before?"

Students love the dashboard. Especially the transcript request, Attendace and Grades feature. No more standing in lines.

Data is safe. Everything works perfectly.

The next chapter explains how we built all of this.

1.4 Project Scope

We had to decide what goes in the system and what stays out. Not everything can be built at once. Some features are in. Some are for later.

The scope is simple. Three types of users get specific features. Nothing more. Nothing less.

Here is what each role can do.

1.4.1 What The Admin Can Do

The administrator owns the system. They see everything. They control everything. But they do not do teaching work. That is for teachers.

Here is the admin's job.

Add and Remove Users. A new student joins the university. The admin opens the panel. Fills a small form. Name. Email. Roll number. Done. The student gets login access. No paper work. No typing in multiple places.

Give Login Access. Every user needs a password. The admin creates these credentials. Students get their own. Teachers get theirs. Other admins get theirs. Each person logs in with their own account. No sharing. No confusion.

Set Up the Academic Year. Before classes start, the admin does the setup. Creates all subjects. Sets up assignment dashboards. Configures exam settings. Generates timetables. Attendance tracking starts automatically.

Without the admin, nothing works. With a good admin, everything flows.

Handle Money and Discipline. Fees are important. The admin sees who paid and who did not. No chasing. No manual lists.

Discipline cases also go through the admin. A teacher reports a student. The admin sees it. Parents get notified. Everything is tracked.

Student requests like transcripts come to the admin. They approve or deny. The student sees the status online. No standing in lines.

1.4.2 What The Teacher Can Do

Here is what we gave them.

See Their Own Schedule. The teacher logs in. Their dashboard shows only their subjects, classes and upload/view assignments. and their timetable. No extra noise.

If a teacher havef three courses, they see three cards. Click on a card. Everything for that course opens.

Mark Attendance. Every classs , the teacher opens their class with in timetable slot otherwise they will not open. A list of student names appears. Click present or absent next to each name. One click per student. The system saves everything.

At the end attendance is already calculated. No manual counting. No registers.

Create Assignments. The teacher clicks "New Assignment." Types a title and description if needed. select a subject and section or in bulk. Sets a due date and add marks of assignment and techer can add Assignment File also. Uploads instructions. Done. All students see it immediately.

No more printing papers. No more handing out sheets in class. No more "I lost the assignment brief."

Upload Grades. Exam time comes. The teacher has a list of student submissions. Opens each one. Types a number. Adds comments. The system calculates final grades automatically.

No Excel sheets. No manual addition. No math mistakes.

The teacher's job becomes teaching. Not paperwork.

1.4.3 What The Student Can Do

Students are the main users. Everything we built is for them.

Here is what they see.

View Their Subjects. The student logs in. A dashboard shows all enrolled subjects. Each subject has its own page. Course materials are there. Assignments are there. Grades are there.

No hunting for files. No asking teachers for notes. Everything is in one place.

Check Timetable and Attendance. The student sees the daily schedule. Which class. Which room. Which teacher. No confusion about where to go.

Attendance records are also visible. How many days present? How many absent? The student knows exactly where they stand. No surprises(short Attendance) at the end of the semester.

Submit Assignments. The student opens an Assignments and click on related subject. Reads the instructions. Uploads their Assignments file. Clicks submit. Done.

The system shows the submission status and Grade. Submitted on time or late? The student sees it. The teacher sees it. No arguments later.

Check Grades. Results come out. The student opens "My Results." Grades are there. Assignment scores. Exam scores. Final totals. No waiting for notice boards. No asking teachers.

Request Transcripts. Need an official document for a job? The student clicks "Request Transcript." Fills a small form. Clicks submit.

The admin sees the request. Approves it. The student downloads the PDF. No standing in lines. No filling paper forms. No waiting for days.

1.4.4 Accountant / Office Clerk Scope

They do not need full admin powers. But they need to handle money, records, and student requests.

Here is what the accountant can do.

Student Management. The accountant can add new students to the system. They fill a form with name, father name, roll number, and contact details. They can also view the complete student list. Search by name or roll number. Edit a student record if something is wrong. No paper forms. No manual files.

Authentication for Students and Teachers. When the accountant adds a new student, the system creates a login account automatically. The student gets an email with their password. Same for teachers. The accountant does not have to set passwords manually. The system handles it.

Subject Management. The accountant can add new subjects to the system. Subject name. Subject code. Credit hours. Department. Semester. Once added, the subject appears for teachers and students.

Exam Dashboard. The accountant sees a complete exam dashboard. Which exams are coming up. Which exams are completed. Student exam entries. The accountant can view but not change exam settings. That is for the admin.

Attendance Dashboard. The accountant can view attendance records for any student or class. But they cannot change attendance. Only view. A teacher might ask "show me who is low on attendance." The accountant pulls the report.

Token Generation. When a student needs to meet someone in the office, the accountant generates a token. A unique number. The student shows the token. The office knows which student and why. No more "who is next?" confusion.

Case Committee. Discipline cases are tracked here. A teacher reports a student. The case goes to the committee. The accountant updates the case status. Open. In review. Closed. Action taken. Parents can be notified. Everything is recorded.

Fee Record Management. This is the accountant's main job. Upload fee records for each student. Amount paid. Date paid. Due date. Remaining balance. The accountant can also view all fee records. Search by student name. See who has not paid. Generate fee reports.

1.4.5 Why We Added This Role

We realized the admin cannot do everything. A university has too many students. Too many records. Too many fees. The admin needs help.

The accountant handles the money side. Student records. Subject setup. Token system. Case tracking. Attendance viewing.

This keeps the admin free for bigger tasks. And the accountant does their job without stepping into teacher or student areas.

1.4.6 Updated Summary Of All User Roles

Now we have four user roles.

Admin. Full control. Everything on the system.

Accountant / Clerk. Manages students, subjects, fees, tokens, cases. Views attendance and exams.

Teacher. Manages courses, assignments, attendance marking, grading.

Student. Views courses, submits assignments, checks grades, view attendance and send request for transcript.

1.4.7 What We Did Not Include

Some things are not in this version.

No mobile app. The website works on phones. But there is no separate app in Google Play.

No automatic grading for multiple choice questions. Teachers must grade everything by hand.

CHAPTER 2
REQUIREMENTS SPECIFICATIONS

2.1 Existing System

Before we built EduSphere, we spent time looking at how things work right now at University of Swabi. Not how they should work. How they actually work every day.

2.1.1 How The Current System Functions

Right now, there is no single system. Teachers use different methods. Some use WhatsApp groups to share notes. A teacher create a group. Adds all students. Sends a PDF. Twenty students say thank you. Ten students never see the message.

Some teachers use email. They attach files. Send to each student individually. That takes hours.

Attendance is done on paper. Marks absent for missing names. At the end of the month, the teacher counts again. Writes totals in another book. If a student asks about attendance, the teacher opens the attendance sheet. Counts again. Time wasted.

Assignments are submitted in many ways. Some students print and hand over physically. Some send files by email. Some bring USB drives. The teacher collects from three different places. Papers get mixed. Files get lost in spam folders. USBs get viruses.

Grades are recorded in Excel sheets. Each teacher has their own file. Different formats. One teacher uses red for fail. Another uses red for pass. No standard. At the end of the semester, all files go to the office. Someone combines them manually. Errors happen.

Transcript requests take days. A student needs to write application for Transcript. Goes to the office. Stands in line.. Comes back after three days. Stands in line again. Gets told "come tomorrow." Comes again. Finally gets the document. One transcript takes one week.

2.1.2 What Does Not Work

The drawbacks are serious.

First, no central view. The admin cannot see what is happening. Which teacher is active? Which student is failing? Which class has low attendance? No one knows. All information is scattered.

Second, students suffers. They never know their status. Attendance percentage? Ask the teacher. Assignment grade? Ask the teacher. Exam result? Wait for the notice board. Everything is a chase.

Third, manual work takes too much time. Teachers spend hours on attendance and grading. Hours that could be used for teaching. Hours that could be used to help

students.

Fourth, errors are common. A teacher adds marks wrong. A student gets lower grade. Complains. Teacher checks. Finds mistake. Fixes it. Wastes everyone's time.

These problems are not small. They affect real people. Real students lose real grades. Real teachers waste real hours.

2.2 Proposed System

EduSphere is our answer to these problems.

2.2.1 Why We Chose This Solution

We looked at other options. Moodle is powerful but too complex. Teachers need training to use it. Google Classroom is simple but does not cover all the requirements. We needed something simple. Something free. Something that works on any phone or computer.

We decided to build our own.

EduSphere is a web/Mobile based application. No software to install on any computer. Open a browser. Type the address. Log in. Done. Works on Chrome, Firefox, Edge, Safari. Works on laptop, tablet, phone.

The interface is clean. No extra buttons. No confusing menus. A teacher who only knows WhatsApp can learn EduSphere in ten minutes.

Data is centralized. One database. One source of truth. No more scattered files. No more lost records.

2.2.2 Other Solutions We Considered

We looked at three other options before deciding to build our own.

Moodle. Open source. Free. Many features. But too heavy. Takes time to set up. Teachers get lost in menus. Nobody used it after one month.

Google Classroom. Simple. Clean. Works well. but does not over all the things which we want. Parents cannot access easily. Also, Google owns the data. The university wanted control.

Canvas. Professional. Feature rich. But expensive. Also, hosted on US servers. Data privacy concerns. None of these worked for us. So we built EduSphere.

2.2.3 Technical Feasibility

Can we actually build this? Yes.

We used Python and Django for the backend. Django is a high level web framework. It is secure. It is fast. It handles database operations easily. Many large websites use Django.

For the frontend, we used HTML, CSS, and JavaScript. These are standard web technologies. Every browser understands them.

We used Bootstrap for styling. Bootstrap makes the website look good on phones and computers. No need to write custom CSS for everything.

Git helps us track code changes. Every time we add a feature, we save a version. If something breaks, we go back to an older version. GitHub stores our code online. Both team members can work on the same code from different places.

All tools are free. Python is open source. Django is free. Bootstrap is free. Git and GitHub have free plans. No licensing costs.

2.2.4 Economic Feasibility

Can anyone afford this? Yes.

No software licenses to buy. No monthly fees. No per student charges.

We built EduSphere using free tools. The instuate already has computers. No new hardware needed.

For deployment, we used PythonAnywhere. Our project is live at `https//EDUSPHEREs.pytho`
The university can use this free plan.

Maintenance requires one person. Half day per week.

2.2.5 Social Feasibility

Will people use it? Yes.

We asked students. They said "yes, please give us online system." We asked teachers. They said "yes, please stop manual system." We asked the admin. He said "yes, please organize everything."

The system is simple. No training needed. A five minute video explains everything.

Students already use phones and laptops. They already browse the web. EduSphere feels familiar.

Teachers already use WhatsApp and email. EduSphere is not harder than those apps.

2.2.6 Minimum System Requirements

To run EduSphere, the department/instituate needs the following.

For Users (Students, Teachers, Admin):

- Any computer or smartphone
- Modern web browser (Chrome, Firefox, Edge, Safari)
- Internet connection
- No software installation needed

For Development (What We Used):

- Python 3.9 or higher
- Django framework
- HTML, CSS, JavaScript for frontend
- Bootstrap 5 for styling

- Git for version control
- GitHub for remote code storage

For Deployment:

- PythonAnywhere cloud platform
- Live URL: `https://EDUSPHEREs.pythonanywhere.com/`

2.3 Functional Requirements

Functional requirements are things the system must do. Features it must have. Actions it must support.

2.3.1 For Students

A student should be able to do these things.

- student have send random password through Email from admin side
- Log in with email and password
- forget/reset password
- View all enrolled courses on dashboard
- View daily timetable with room numbers
- Check attendance percentage anytime
- See upcoming assignments with due dates
- Submit assignment files online
- View grades after teacher releases them
- Request official transcript online
- Track transcript request status
- Change password

2.3.2 For Teachers

A teacher should be able to do these things.

- Teacher have send random password through Email from admin side
- Log in with email and password
- teacher set their availability time slot to the concen department
- View their timetable with the availble time slot.
- View assigned subjects on dashboard
- Upload course materials (PDF, DOC, PPT)
- Mark student attendance digitally
- View attendance reports for each class
- Create new assignments with due dates
- Upload assignment instructions
- View student submissions in one list
- Grade each submission with score and feedback
- Release grades to students
- View class performance reports
- Update profile and password

2.3.3 For Administrator

The admin should be able to do these things.

- Log in with email and password
- View complete system dashboard
- Add new students (name, email, roll number)
- Add new teachers (name, email, department)
- Remove or deactivate users
- Create new subjects and courses
- Assign teachers to subjects

- Enroll students in subjects
- Generate class timetables
- View all attendance records
- View all grades across all subjects
- Track fee payment status
- Review and approve transcript requests
- Generate system reports (PDF or Excel)
- Backup database
- Change system settings

2.3.4 For Accountant / Office Clerk

The accountant should be able to do these things.

Student Management:

- Add new student to the system (name, father name, roll number, email, phone, address)
- View complete list of all students
- Search students by name or roll number
- Edit student information when needed
- Deactivate a student who has graduated or left

Authentication Management:

- System automatically creates login for each new student
- System automatically creates login for each new teacher
- Reset student password if forgotten
- Reset teacher password if forgotten
- View list of active and inactive accounts

Subject Management:

- Add new subject (name, code, credit hours, department, semester)

- View all subjects in the system
- Edit subject details
- Assign subject to a teacher
- Enroll students in a subject

Exam Dashboard:

- View upcoming exam schedule
- View completed exam list
- See student exam entries
- Generate roll number slips for students
- Note: Accountant can only view exams, not create or edit

Attendance Dashboard:

- View attendance records for any student
- View attendance summary for any class
- Generate attendance report (weekly, monthly, semester)
- Note: Accountant cannot mark or edit attendance, only view

Token Generation:

- Generate a new token number for student visit
- Token includes student name, roll number, purpose, date, time
- View all active tokens
- Close token when service is completed
- Search token history by student name or date

Case Committee / Discipline:

- Create a new case when a complaint is received
- Case fields: student name, reporter name, description, date
- Update case status (Open, Investigation, Review, Closed)
- Add committee remarks to a case

- View all cases with filters (open, closed, by student)
- Notify parents when a case is opened or resolved
- Generate case report for committee meetings

Fee Record Management:

- Upload fee payment for a student
- Fee fields: amount paid, date, payment method, receipt number, due date
- View complete fee history for any student
- See pending fees list (students who have not paid)
- Edit a fee record if mistake was made
- Delete incorrect fee entry (with admin approval)
- Generate fee report for a class or department
- Export fee data to Excel for audit purposes

2.3.5 Updated Use Cases For Accountant Role

Use Case 7: Accountant Adds a New Student

A new student named Bilal gets admission. He came to the office. The accountant opens EduSphere. Click "Add Student." Fills the form. Name, father name, roll number, Phone number, and Email address. Clicks save. The system creates Bilal's login account automatically. An email goes to Bilal with his password. Total time? Two minutes.

Use Case 8: Accountant Records Fee Payment

Bilal pay the semester fee. 65,000 rupees. The accountant opens EduSphere. Searches for Bilal. Click "Add Fee." Enters amount, date, payment method. Uploads receipt scan. Clicks save. The fee record is saved. Bilal can now see his fee status when he logs in.

Use Case 9: Accountant Generates a Token

Bilal comes to the office. He needs a transcript. The accountant generates a token. Number 104. The system shows "Token 104 - Bilal - Transcript Request - 10:30 AM." Bilal waits. When his turn comes, the clerk calls "Token 104." Bilal goes to the counter. The clerk closes the token. Done. No crowd. No pushing. No "who was next?"

Use Case 10: Accountant Opens a Discipline Case

A teacher reports that two students were fighting. The accountant opens EduSphere. Clicks "New Case." Selects the student names. Writes the complaint. Sets status to

"Open." The system notifies the committee. The committee reviews. Updates status to "Review." Adds remarks. Finally closes the case with action taken. Everything is recorded. No paper. No lost files.

Use Case 11: Accountant Views Attendance Report

A teacher wants to know which students have low attendance. The accountant opens EduSphere. Goes to Attendance Dashboard. Selects the class. Selects date range. Clicks "Generate Report." The system shows attendance percentage for each student. The teacher sees who is below 75 percent. Takes action. No manual counting.

2.3.6 Updated Summary of All Functional Requirements

Now we have four user types with clear responsibilities.

Admin: Full system control. User management. Course creation. Report generation.

Accountant: Student management. Authentication. Subjects. Exam view. Attendance view. Tokens. Cases. Fee records.

Teacher: Course content. Attendance marking. Assignments. Grading.

Student: View courses. Submit work. Check grades. Request transcripts.

The system is complete. No missing pieces.

2.4 Non-Functional Requirements

Non-functional requirements are about quality. How well the system performs. How safe it is. How easy to use.

2.4.1 Security

Passwords must be hashed. Not stored as plain text. Django handles this by default. No one should see another user's password.

Only logged in users can access the system. No public access to dashboards.

Each user sees only their own data. A student cannot see another student's grades. A teacher cannot see another teacher's classes.

2.4.2 Performance

Page load should take few seconds. Even on slow internet.

The system should handle 500 users at once. University has many students. All may log in during exam season. Django can handle this with proper database optimization.

2.4.3 Usability

A new user should understand the system in five minutes. No training manual needed.

Buttons must be clearly labeled. No confusing icons.

Menus should be simple. A student should not see teacher options. A teacher should not see admin options.

2.4.4 Availability

The system should be available 24/7. Students study at night. They need access.

PythonAnywhere offers 99.9 percent uptime. The free plan has some limits. But for university use, it works fine.

2.4.5 Maintainability

Code must be clean. Comments must explain complex parts. Git tracks all changes. If a bug appears, we can see what changed and fix it.

Django makes maintenance easy. The framework handles many common tasks automatically.

2.5 Use Cases

Use cases show how the peoples will use EduSphere.

2.5.1 Use Case 1: Student Submits an Assignment

A student named Ali logs into EduSphere. He goes to his dashboard. He sees a list of assignments. One assignment is due tomorrow. He clicks on it. He reads the instructions. He uploads his PDF file. He clicks submit. The system shows "Submission Successful." Ali is happy. He does not have to print anything. He does not have to go to the teacher's room.

2.5.2 Use Case 2: Teacher Grades Submissions

A teacher named Sana logs into EduSphere. She goes to her course page. She clicks "Pending Submissions." She sees a list of 25 students. She opens the first submission. She reads it then types 85 in the grade box. She write "Good work, but check grammar." then click save. The student sees the grade immediately. No paper. No Excelsheet files. No manual addition.

2.5.3 Use Case 3: Student Requests Transcript

A student named Fatima needs a transcript for a job interview. She logs into EduSphere. She clicks "Documents." She clicks "Request Transcript." She fills a small form. then click submit. The admin gets a notification. The admin approves it. Fatima gets an email. She logs in and downloads her transcript. No standing are waiting. No waiting for days. No paper forms.

2.5.4 Use Case 4: Admin Adds New Course

The admin logs into EduSphere. He click "Courses." He click "Add New Course." He types "Computer Networks" as the name. then select "Dr. Ahmed" as the teacher. He select the semester. He click save. The course appears for all enrolled students. No paperwork. No forms. No delays.

2.5.5 Use Case 5: Teacher Marks Attendance

A teacher logs into EduSphere. He opens his morning class. A list of student names appears. He click "Present" next to each name. One student is absent. He clicks "Absent" next to that name. then click "Save Attendance." The system updates the attendance records. No register. No manual counting. No mistakes.

2.5.6 Use Case 6: Student Checks Grade

A student logs into EduSphere after exams. He goes to "My Results." He sees his grades for all subjects. He sees assignment scores and exam scores. He sees the final percentage. He does not have to wait for the notice board. He does not have to ask the teacher. He just looks.

2.5.7 Summary

Something has to change.

EduSphere is that change. We built it as a web app. Python and Django do the heavy lifting in the back. You can see it live at EDUSPHEREs.pythonanywhere.com. Go check it out.

What can people do on EduSphere? Students submit work and see scores. Teachers take attendance and give grades. The admin sees everything and controls the rest.

We also made sure the system is safe. It loads fast. Anyone can figure it out in a few minutes. No training class needed.

The use cases we just walked through? Those are real. That is how people will use this system every day.

Up next is Chapter Three. That is where I show the database design. The screens you will see. How all the pieces fit together.

CHAPTER 3

SYSTEM DESIGN

3.1 Architecture Design

Before we wrote any code, we planned how everything would fit together. A house needs a blueprint. Software needs architecture. Here is ours.

3.1.1 Basic System Components

EduSphere has three main parts.

The Frontend. This is what users see. Login page. Dashboard. Assignment form. Grade view. We built it with HTML, CSS, and JavaScript. Bootstrap makes it look good on phones and computers.

The Backend. This is the brain. It handles login. Saves data. Processes forms. Sends emails. We used Python and Django. Django is fast and secure.

The Database. This is where data lives. Student names. Teacher details. Assignment files. Grade numbers. We used MySQL. It is reliable and free.

These three parts talk to each other. User clicks a button. Frontend sends a message to backend. Backend asks database. Database gives answer. Backend sends response to frontend. User sees result. All in less than three seconds.

3.1.2 Type of Architecture Selected

We chose Model-View-Template architecture. Django uses this by default.

Model. This talks to the database. When we need student data, the model fetches it. When we save a grade, the model stores it.

View. This contains the logic. What happens when a user logs in? What happens when a teacher clicks "Create Assignment"? The view decides.

Template. This is the HTML page the user sees. Django fills it with data from the view.

This architecture is clean. Each part does one job. If we need to change the database, we do not touch the frontend. If we change the design, we do not break the logic.

3.1.3 High Level System Model

Here is how data flows.

A student logs in. Their browser sends a request to the Django server. Django checks the database. Finds the student. Create a session. Sends back the dashboard page.

The student click "Submit Assignment." Chooses a file. Click on upload. Django saves the file to the server. Writes a record in the database. Shows "Success" message.

The teacher logs in. Opens the assignment. Sees all submissions. Click on a student name. Sees their file. Types a grade. Click save. Django updates the database. The

student sees the grade immediately.

Everything happens in seconds. No waiting. No delays.

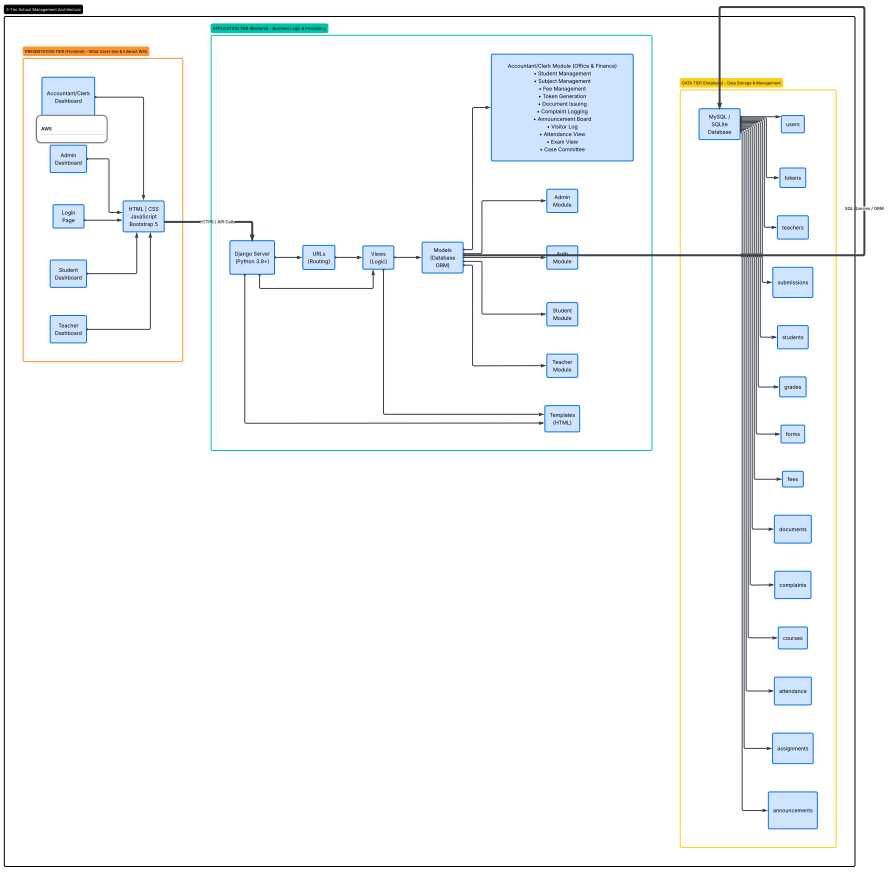

Figure 3.1: EduSphere System Architecture

3.2 Component Level Design

We broke the system into small pieces. Each piece does one job. This makes the code easy to write and fix.

3.2.1 Authentication Module

This module checks who is logging in.

When a user types email and password, Django checks the database. If correct, it creates a session. The user stays logged in until they logout or close the browser.

Each user has a role. Student. Teacher. Accountant. Admin. The system remembers the role. It shows only what that role should see.

Files: views.py (login function), urls.py, login.html

How it works: User submits form. Django validates. Redirects to dashboard.

Error handling: Wrong password shows message. Too many failed attempts locks account for 5 minutes.

3.2.2 Student Module

This module gives students what they need.

Students see their courses. Download materials. Submit assignments. Check grades. Request transcripts.

Each student only sees their own data. No peeking at other students.

Files: student_dashboard.html, submit_assignment.html, view_grades.html

Data tables used: students, enrollments, assignments, submissions, grades

Error handling: If no file uploaded, show error. If file too big, reject. If late submission, mark as late.

3.2.3 Teacher Module

Teachers manage their courses.

They upload materials. Create assignments. Mark attendance. Grade submissions. Give feedback.

Each teacher only sees their own classes. Not other teachers' work.

Files: teacher_dashboard.html, create_assignment.html, mark_attendance.html, grade_submission.

Data tables used: teachers, courses, assignments, submissions, attendance, grades

Error handling: If due date is past, show warning. If no students in class, show message.

3.2.4 Accountant Module

The accountant handles the office side.

They add students. Manage subjects. Upload fees. View attendance. View exams. Generate reports.

The accountant cannot change attendance or grades. Only view.

Files: accountant_dashboard.html, add_student.html, manage_subjects.html, upload_fee.html, view_reports.html

Data tables used: students, subjects, fees, enrollments

Error handling: If student already exists, show error. If fee amount is negative, reject.

3.2.5 Admin Module

The admin controls everything.

They add users. Create courses. Assign teachers. Enroll students. Change settings.

The admin sees all data. No restrictions.

Files: admin_dashboard.html, manage_users.html, manage_courses.html, system_settings.html

Data tables used: all tables

Error handling: Cannot delete user with active data. Cannot remove course with enrolled students.

3.2.6 Diagrams

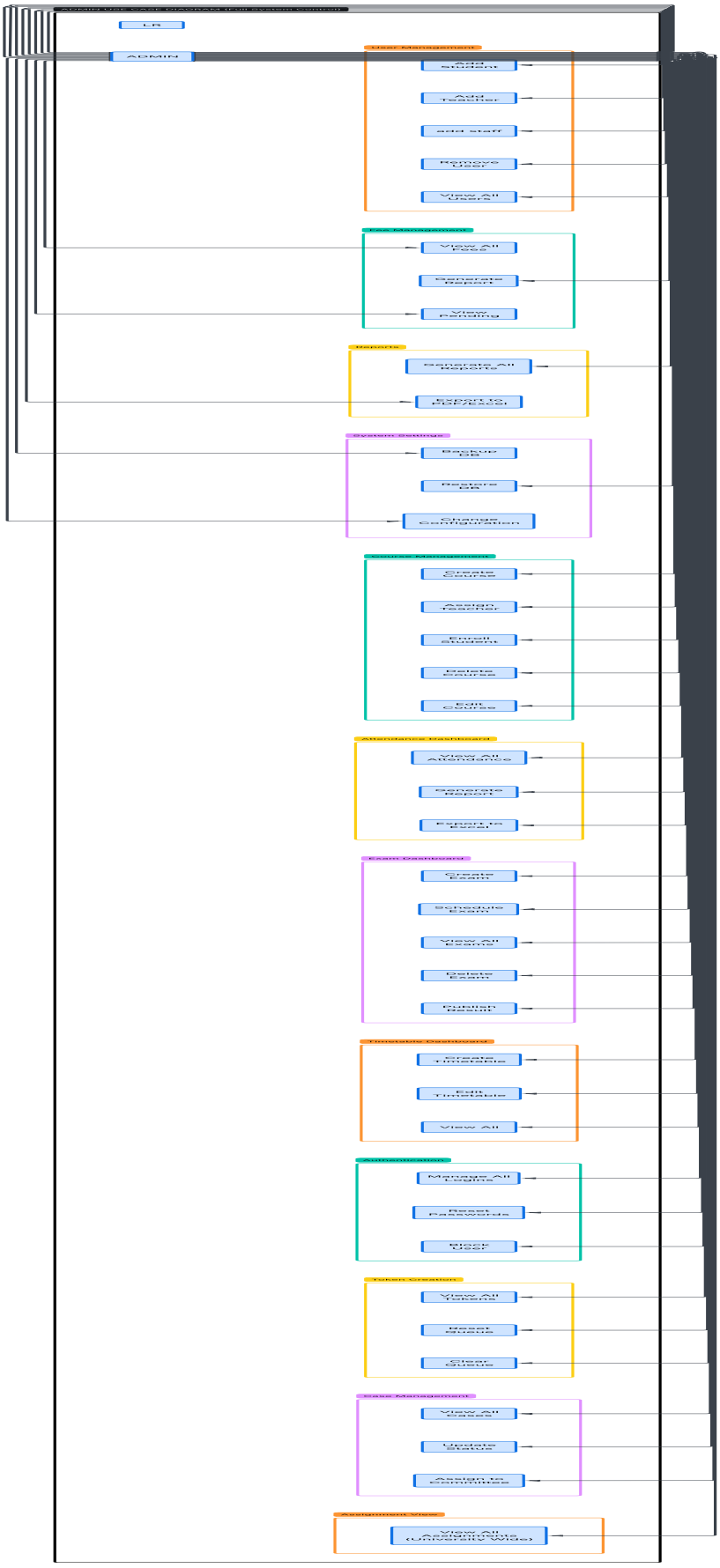

Figure 3.2: Admin Use Case Diagram - Full System Control

Figure 3.3: Accountant/Clerk Use Case Diagram - Office and Finance Management

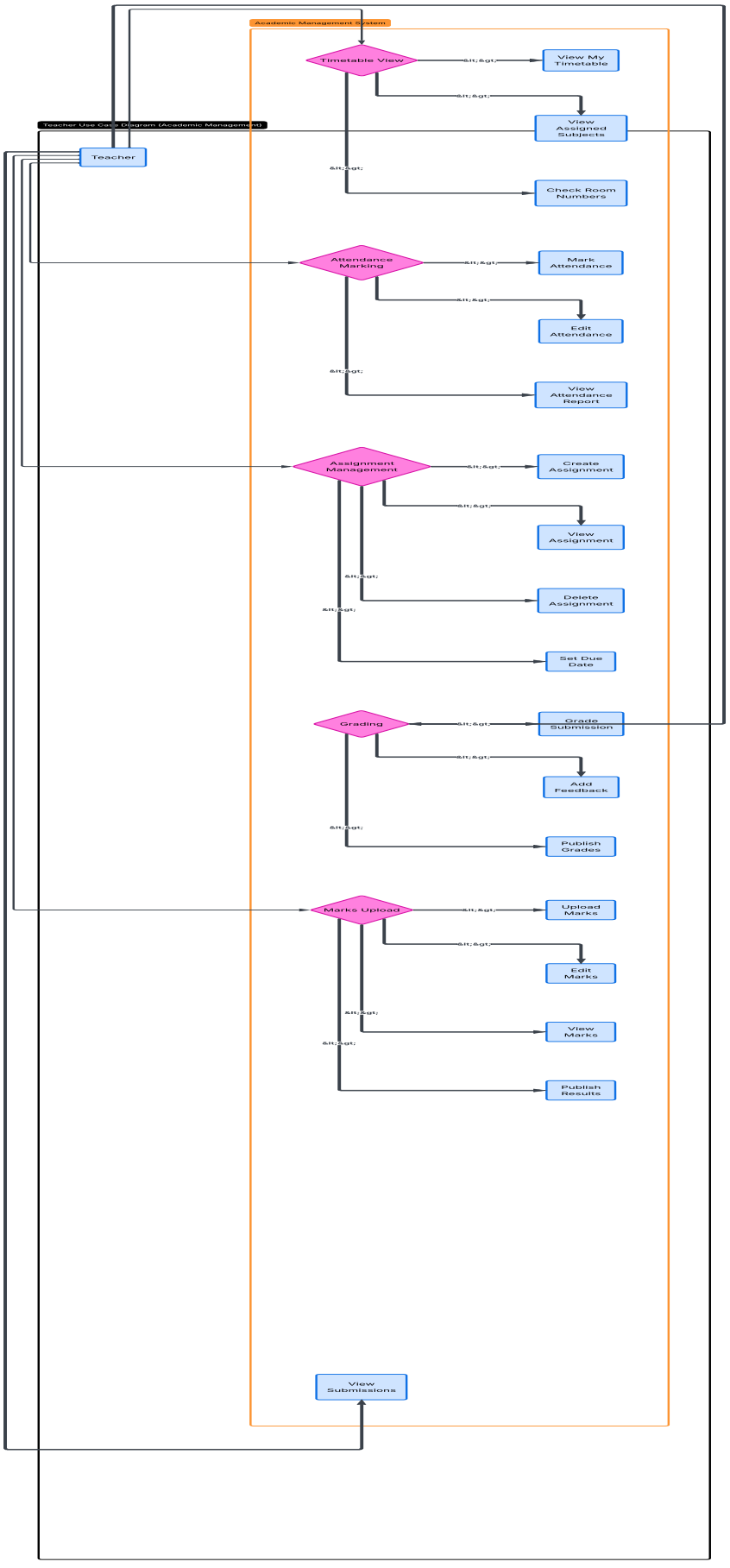

Figure 3.4: Teacher Use Case Diagram - Academic Management

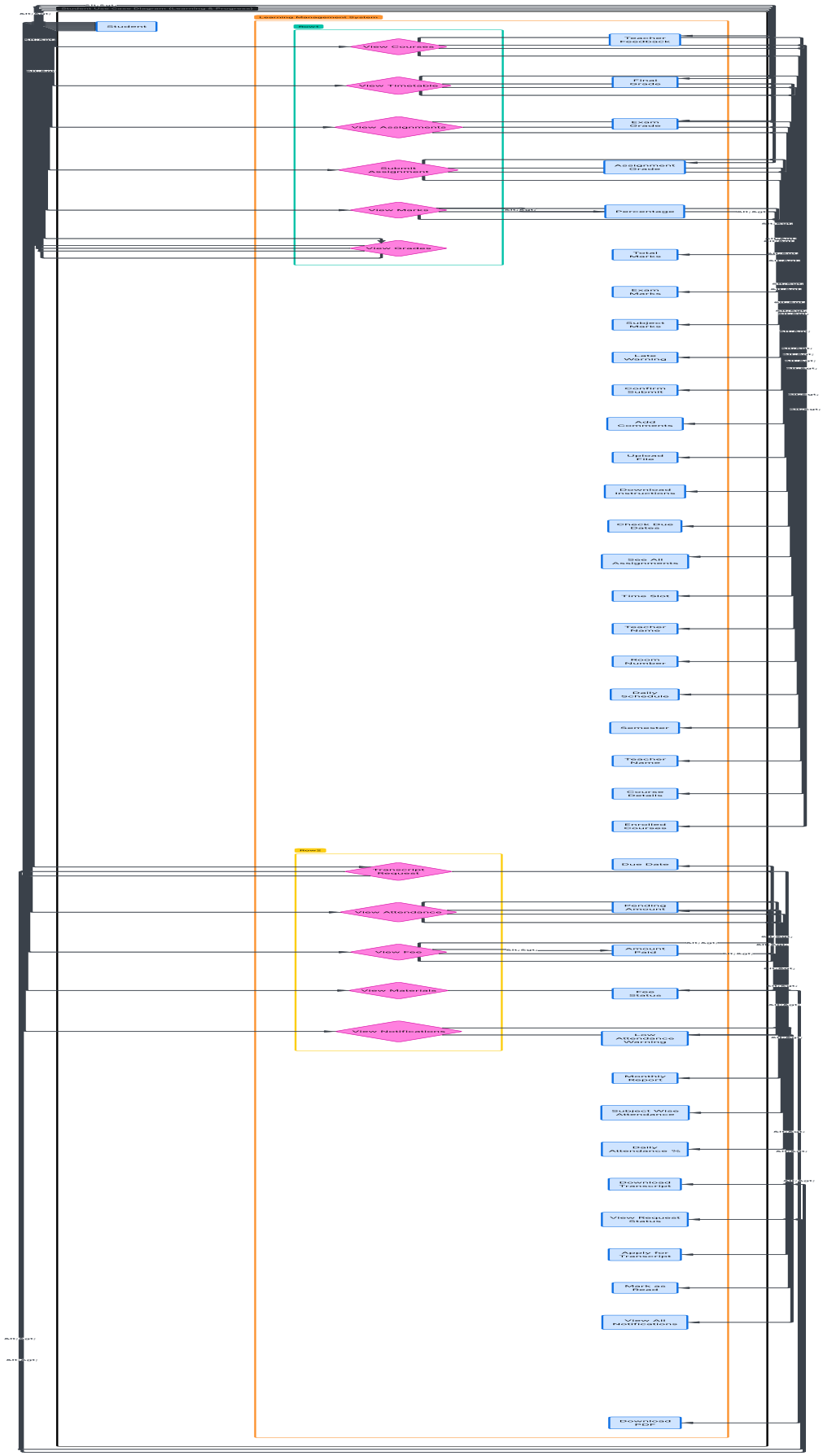

Figure 3.5: Student Use Case Diagram - Learning and Progress

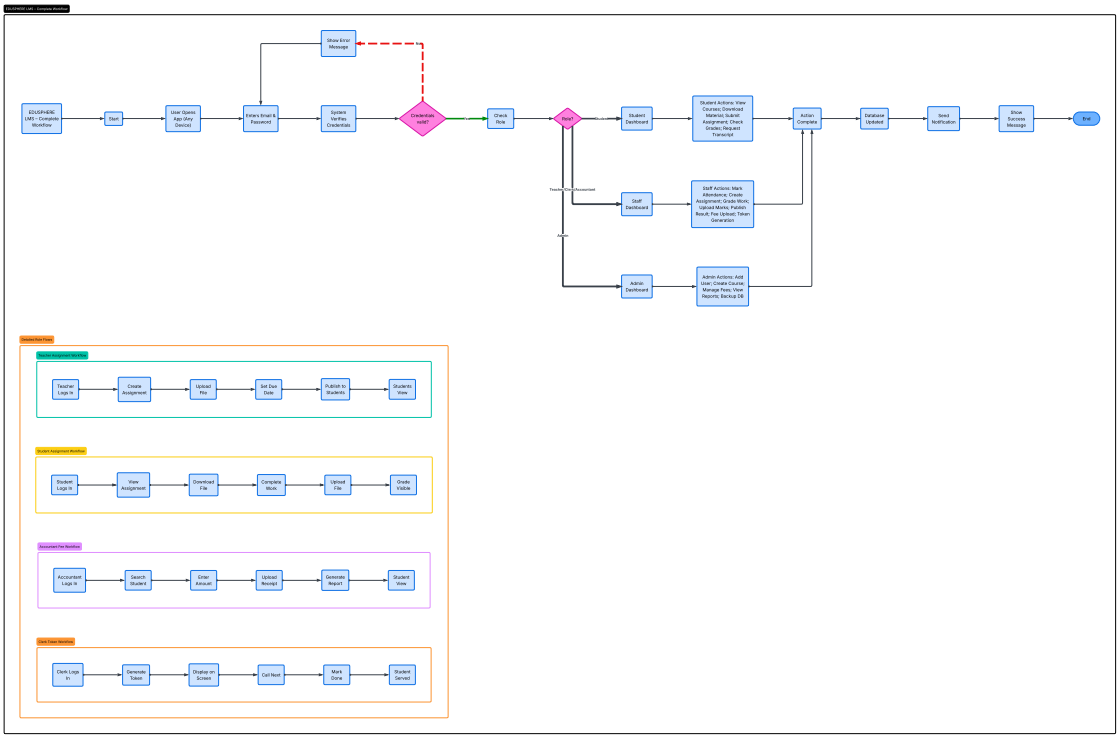

Figure 3.6: Activity Diagram - Complete System Workflow

3.3 Data Design

Data is important for every software without that an software cannot work . If the data is messy, nothing works. We spent time planning every table. Every column. Every connection.

3.3.1 Business Entities

These are the real things we store in the database.

User. Anyone who logs into the system. A student. A teacher. An accountant. An admin. Every user has only one account.

Student. A user who comes to learn. They enroll in courses. They see their timetable and subjects. They submit assignments. They get grades. They check attendance.

Teacher. A user who teaches. They see their subjects and timetable. They view their schedule. They create assignments. They mark attendance. They give grades.

Accountant. A user who handles fee and subjects. They handle add students. They handle add teachers. They view attendance. They check exam dashboards. They manage cases. They create logins for students and teachers.

admin. A user who runs the hole software. who access everything.

Course. One subject taught in one semester. It has one teacher. It has many students enrolled.

Assignment. A task given to students. It has a due date and instructions. student

submit assignment.

Submission. A student's completed work. An uploaded file. The date they submitted. The grade they received. The feedback from teacher.

Attendance. A record of who came to class. The date of class. Which student. Status is present or absent.

Fee. A payment record. The amount paid. The date of payment. Which student paid. Receipt number for tracking.

Document. An official paper from the office. Bonafide certificate. Character certificate. Transcript.

Complaint. A student reporting a problem. Type of complaint. Description of issue. Current status. Resolution details.

3.3.2 Entity Relationship Diagram

The diagram below shows how all these tables connect to each other.

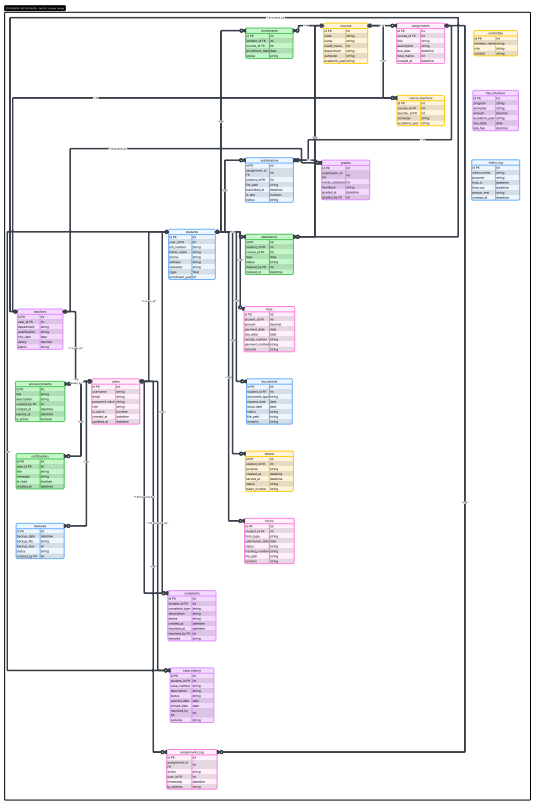

Figure 3.7: Entity Relationship Diagram for EduSphere Database

3.3.3 Data Dictionary

Here are the main tables and what each column means.

3.3.3.1 users table

This table stores login information for every person who uses the system.

- **id** - A unique number for each user. The system uses this to find people.
- **username** - The name of the user.
- **email** - Their email address. Used for login and sending notifications.
- **password** - The password are store in hashing. Never stored as plain text.
- **role** - What type of user. Student, teacher, accountant , or admin.
- **created_at** - The date and time when the account was made.

3.3.3.2 students table

This table holds extra information about students.

- **id** - A unique number for each student record.
- **user_id** - Links to the users table. One user becomes one student.
- **roll_number** - The university roll number printed on their ID card.
- **father_name** - Student's father name for official records.
- **phone** - Contact number for emergencies.
- **address** - Home address for correspondence.
- **semester** - Current semester number from 1 to 8.

3.3.3.3 teachers table

This table holds extra information about teachers.

- **id** - A unique number for each teacher record.
- **user_id** - Links to the users table. One user becomes one teacher.
- **department** - Which department they work in. CS, English, Math, etc.
- **qualification** - Their highest degree. PhD, MPhil, Masters.
- **hire_date** - The date when they joined the university.

3.3.3.4 courses table

This table stores all the subjects taught at the university.

- **id** - A unique number for each course.
- **code** - Short subject code like CS101 or MATH201.
- **name** - Full course name like "Programming Fundamentals".
- **credit_hours** - How many credits the course is worth.
- **teacher_id** - Which teacher is assigned to teach this course.
- **semester** - Which semester this course belongs to.

3.3.3.5 enrollments table

This table connects students to the courses they take.

- **id** - A unique number for each enrollment record.
- **student_id** - Which student is enrolled.
- **course_id** - Which course they are taking.
- **enrollment_date** - When they registered for this course.

3.3.3.6 assignments table

This table stores all the tasks teachers give to students.

- **id** - A unique number for each assignment.
- **course_id** - Which course this assignment belongs to.
- **title** - Short name like "Assignment 1" or "Mid Project".
- **description** - Full instructions for students.
- **due_date** - Last date and time to submit.
- **total_marks** - Maximum grade possible for this assignment.

3.3.3.7 submissions table

This table stores student work after they upload it.

- **id** - A unique number for each submission.
- **assignment_id** - Which assignment this is for.
- **student_id** - Which student submitted the work.
- **file_path** - Where the file is stored on the server.
- **submitted_at** - Date and time of submission.
- **is_late** - Yes if submitted after due date. No if on time.

3.3.3.8 grades table

This table stores the marks and feedback for each submission.

- **id** - A unique number for each grade record.
- **submission_id** - Which submission is being graded.
- **marks_obtained** - The score the student received.
- **feedback** - Comments written by the teacher.
- **graded_at** - Date and time when teacher graded.

3.3.3.9 attendance table

This table records who came to class each day.

- **id** - A unique number for each attendance record.
- **student_id** - Which student is marked.
- **course_id** - Which class they attended.
- **date** - The date of the class.
- **status** - Present or absent.

3.3.3.10 fees table

This table tracks every payment made by students.

- **id** - A unique number for each fee record.
- **student_id** - Which student paid.
- **amount** - How much money was paid.
- **payment_date** - When the payment was made.
- **due_date** - Last date to pay without fine.
- **receipt_number** - Official receipt number for tracking.
- **status** - Paid or pending.

3.3.3.11 documents table

This table stores requests for official certificates.

- **id** - A unique number for each document request.
- **student_id** - Which student requested the document.
- **document_type** - Bonafide, character, or transcript.
- **request_date** - When the student applied.
- **issue_date** - When the document was given.
- **status** - Pending, issued, or rejected.

3.3.3.12 tokens table

This table manages the queue for office visits.

- **id** - A unique token number.
- **student_id** - Which student took the token.
- **purpose** - Why they came to the office.
- **created_at** - When the token was generated.
- **served_at** - When the student was called.
- **status** - Waiting, served, or cancelled.

3.3.4 How Tables Connect

The users table connects to students and teachers. One user becomes one student. One user becomes one teacher. The students table connects to enrollments. One student can take many courses. The courses table connects to enrollments. One course can have many students. The courses table connects to assignments. One course can have many assignments. The assignments table connects to submissions. One assignment can have many submissions. Submissions connects to grades. One submission has one grade. The students table connects to attendance. student has many attendance records. The students table connects to fees. Student has many fee payments. The students table connects to documents. student can request many documents. The students table connects to tokens. Student can get many tokens over time. This design keeps data organized. No repetition. No confusion. Everything has its place.

3.4 User Interface Design

The interface must be simple. No confusing buttons. No hidden menus.

3.4.1 UI Design Decisions

We made these choices before building.

Clean and simple. No extra decorations. No flashy colors. Just what users need.

Blue and white. Blue is calm. White is clean. Easy on the eyes.

Responsive. Works on phone. Works on computer. Same experience everywhere.

Consistent. Same menu style on every page. Users never get lost.

3.4.2 Design Tools Used

We used Figma to draw the screens first. Then we built them with HTML, CSS, and Bootstrap.

Bootstrap is a framework. It gives us ready made buttons and menus. We do not have to write everything from zero.

3.4.3 Screen Layouts

Here are the main screens in EduSphere.

Login Page. Two fields. Email and password. One button. Login. Simple.

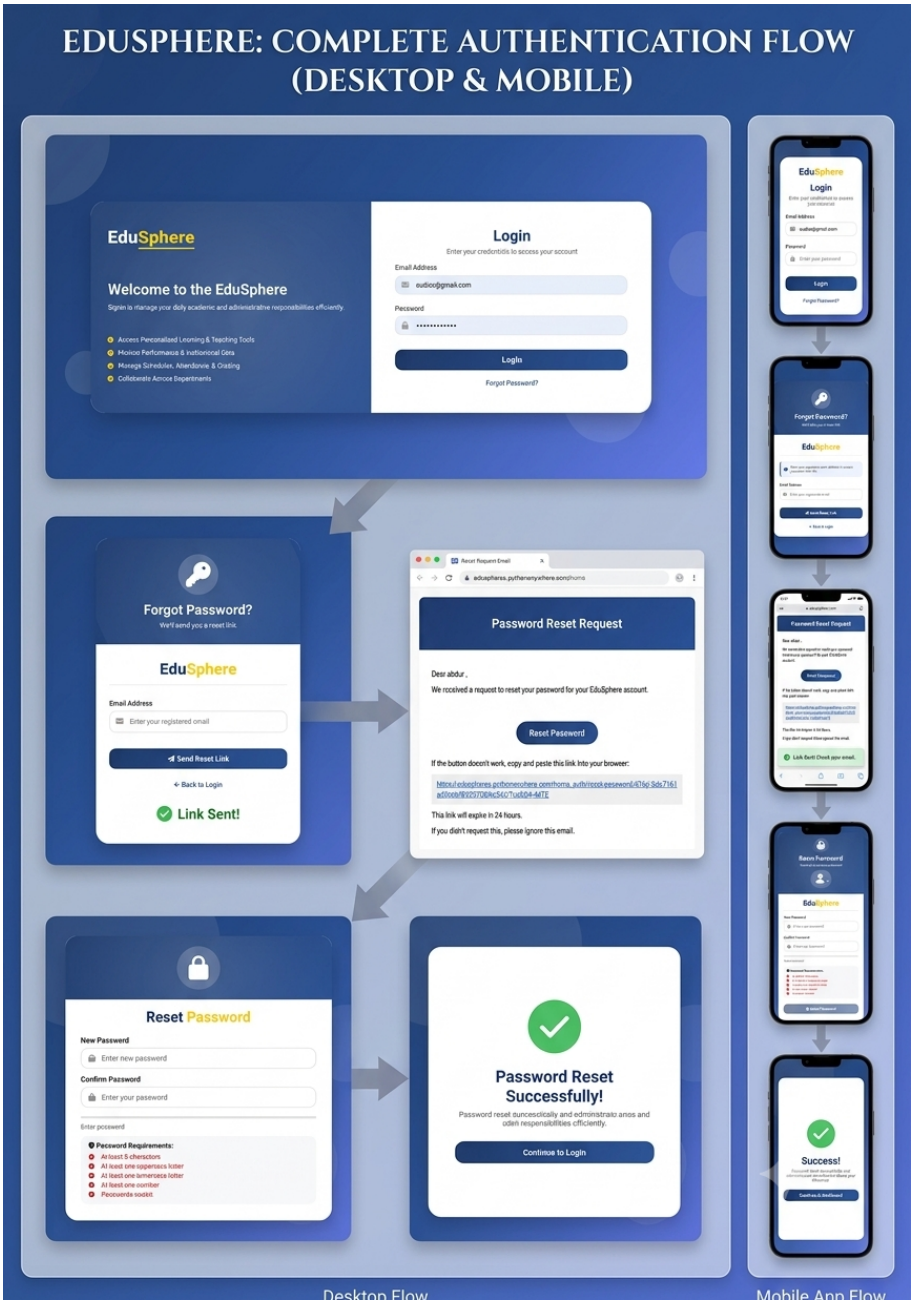

Figure 3.8: Login Page - mobile/desktop

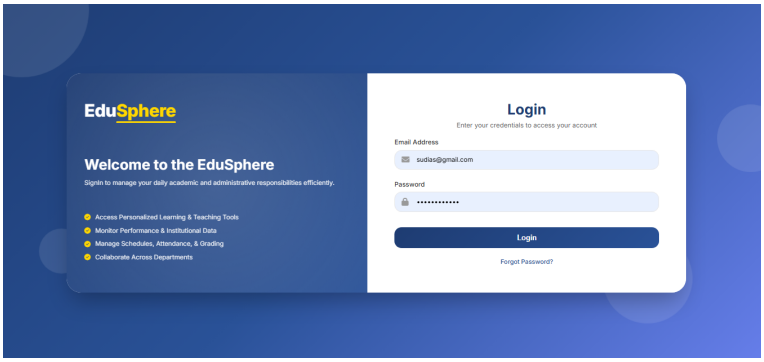

Figure 3.9: Login Page Interface

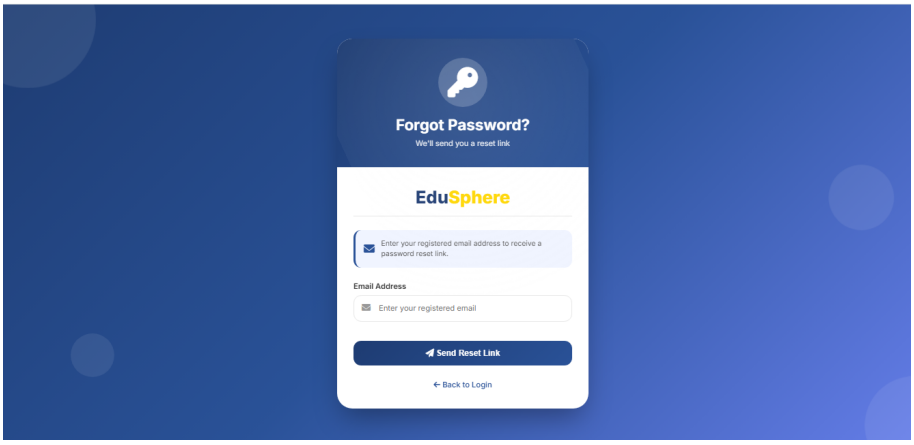

Figure 3.10: Forgot Password Page - Email Submission

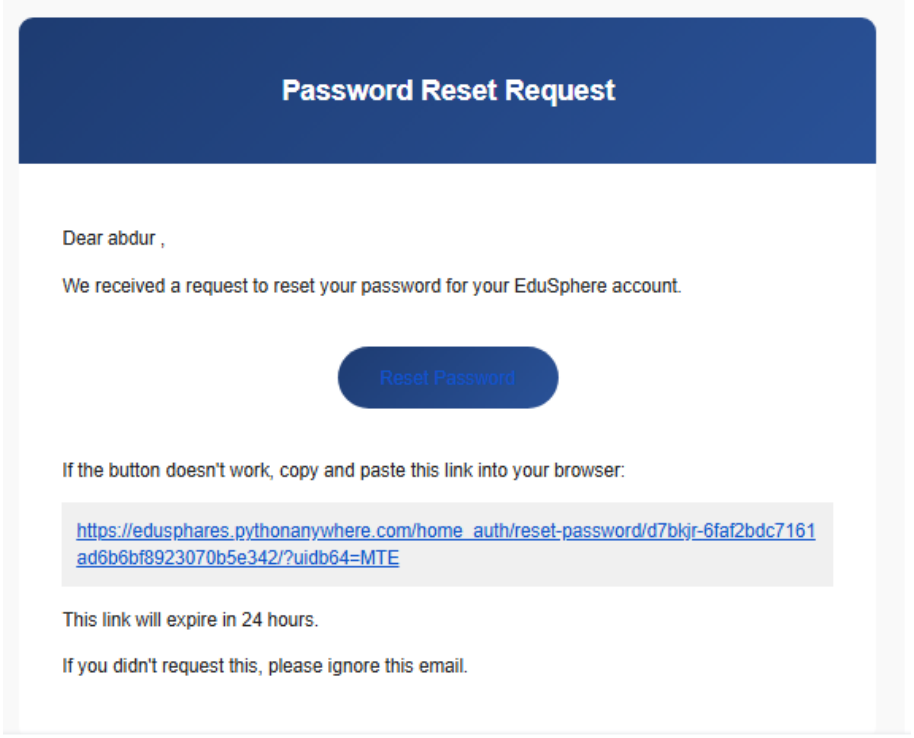

Figure 3.11: Password Reset email Link Sent Confirmation

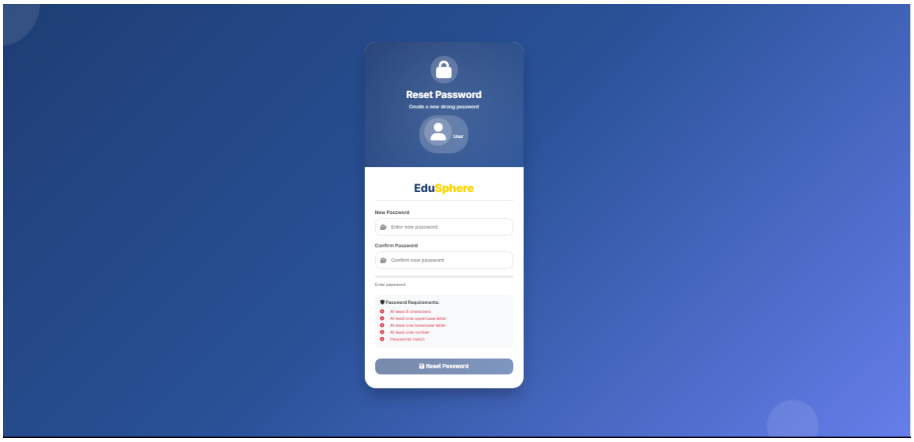

Figure 3.12: Reset Password Form - Enter New Password

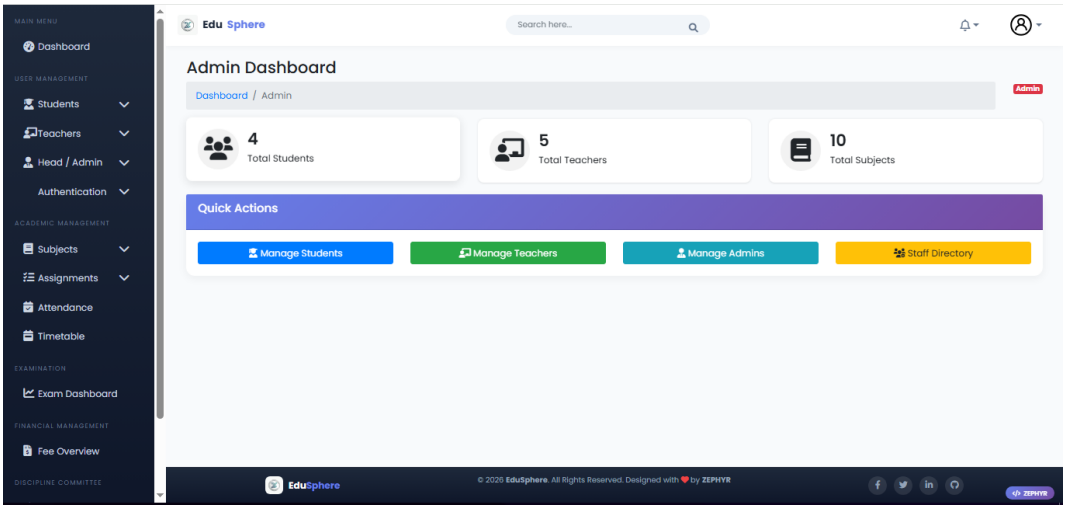

Figure 3.13: Admin Dashboard - Overview and Statistics

What this screen shows. This is where the admin lands after logging in. The big numbers sit at the top. Total students. Total teachers. Total courses. Pending fees. The admin sees everything in one place. Recent activity feeds show who joined, who submitted work, who complained. Buttons for common tasks are right there. Add a user. Create a course. The sidebar on the left has links to every section. User management. Courses. Attendance. Exams. Fees. Settings. No need to click around too much.

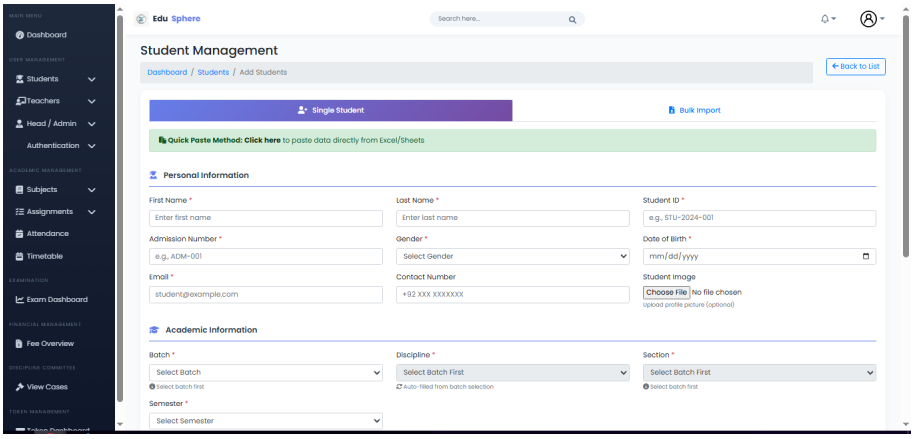

Figure 3.14: Add Single Student - Manual Entry Form

Adding one student at a time. Sometimes you just need to add one student. This form does that. Fields ask for name, father name, roll number, email, phone, address, semester, and department. Hit submit. The system checks if anything is missing or wrong. Then it makes a login account and emails the student their password. Good for when bulk upload is too much. Maybe one student joined late. Maybe a transfer student came in. This handles that.

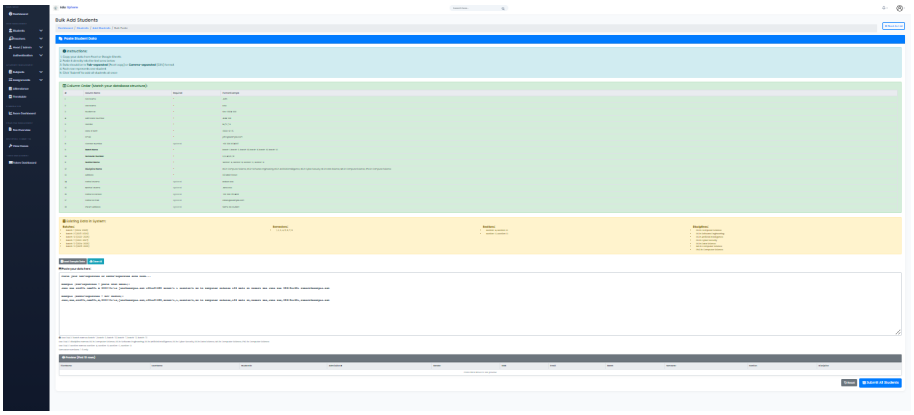

Figure 3.15: Bulk Student Upload - CSV and Excel File Import

Adding many students at once. At the start of a semester, hundreds of students join. Typing each one by hand takes days. This screen fixes that. Download a template file. Fill in the rows. Upload back to EduSphere. The system reads every row. Creates accounts for everyone. If something is wrong in row 45, it tells you exactly where the problem is. Fix it. Upload again. Hundreds of students added in minutes instead of days.

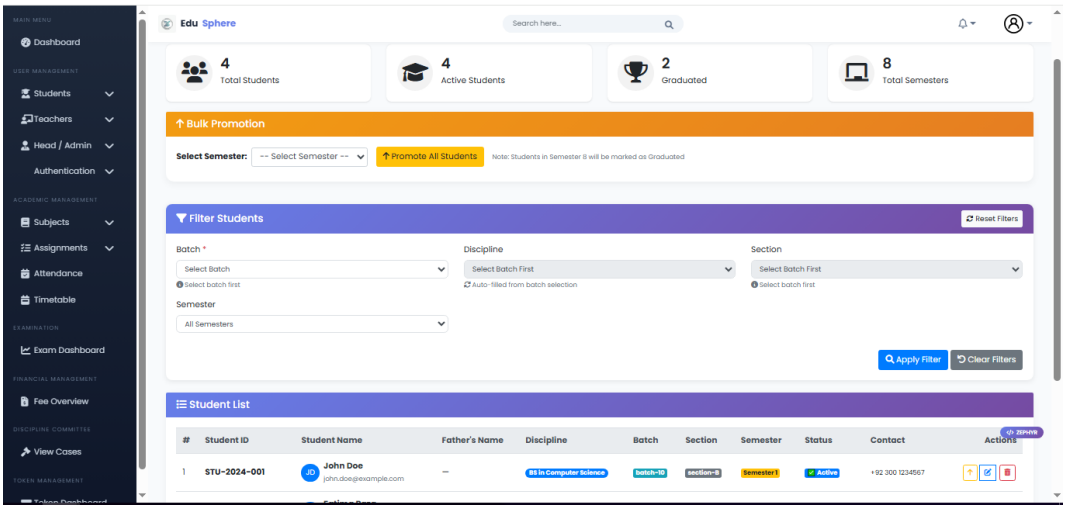

Figure 3.16: Student List - View, Search, Edit, Delete Options

Seeing all students. Every student in the university appears here in a table. Roll number. Name. Father name. Email. Phone. Semester. Status. The admin can search by name or roll number. Filters help narrow down by semester or department. Next to each student there are buttons. Edit. Delete. Reset password. View full profile. Clicking view details shows everything. Enrolled courses. Attendance records. Grades. Fee history. The admin can also download the whole list as PDF or Excel. Good for keeping offline copies.

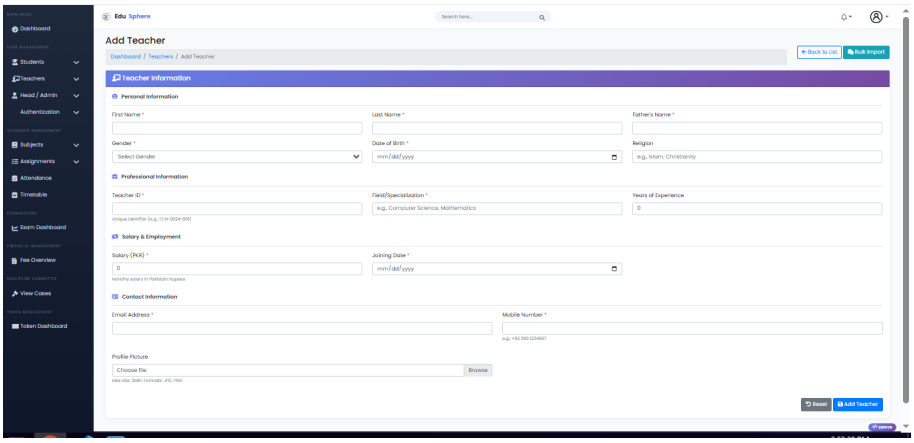

Figure 3.17: Add Single Teacher - Manual Entry Form

Add one teacher. Fill the name, email, department, qualification, and hire date. Click save. System makes a login account. Teacher gets email with password.

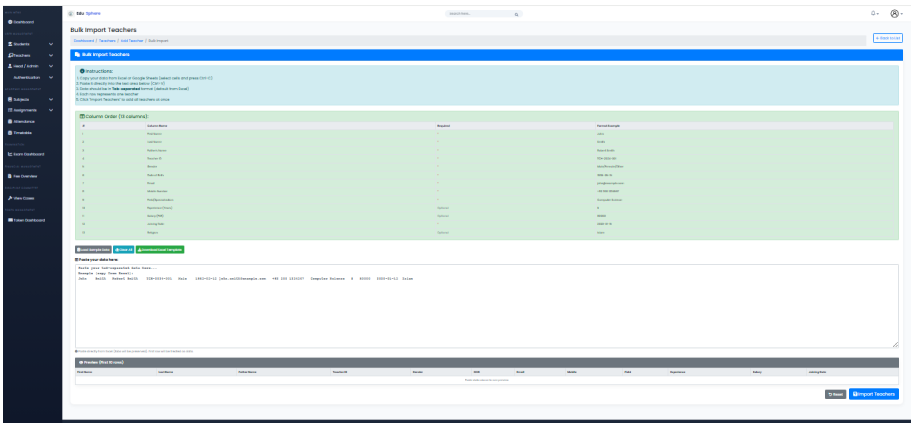

Figure 3.18: Bulk Teacher Upload - CSV File Import

Add many teachers at once. Download the CSV template. Fill teacher data in Excel. Upload the file. System reads every row. Creates accounts for all. Saves hours of typing.

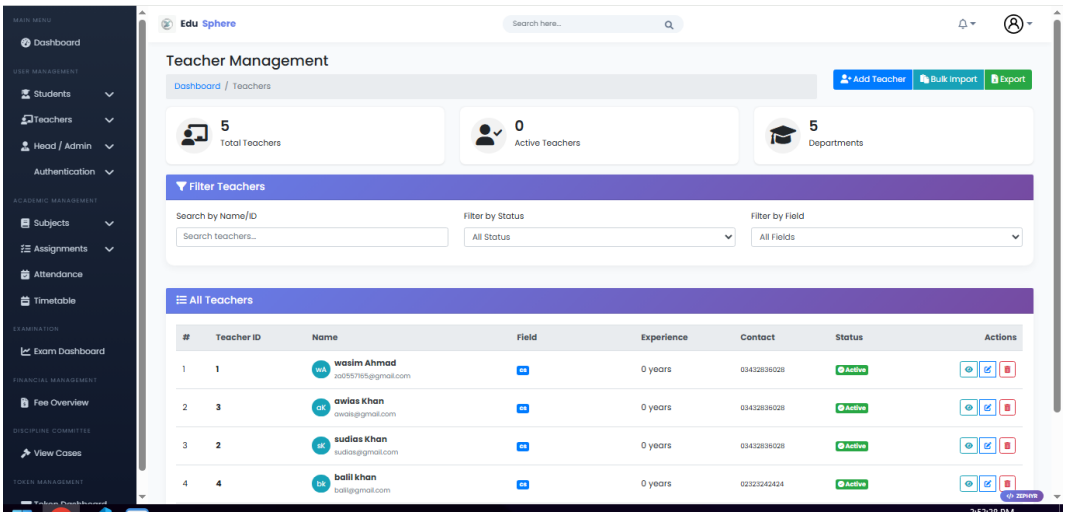

Figure 3.19: Teacher List - View, Search, Edit, Delete

All teachers in one table. See name, email, department, qualification. Search by name. Filter by department. Edit or delete any teacher. Reset password if needed. Export list to Excel.

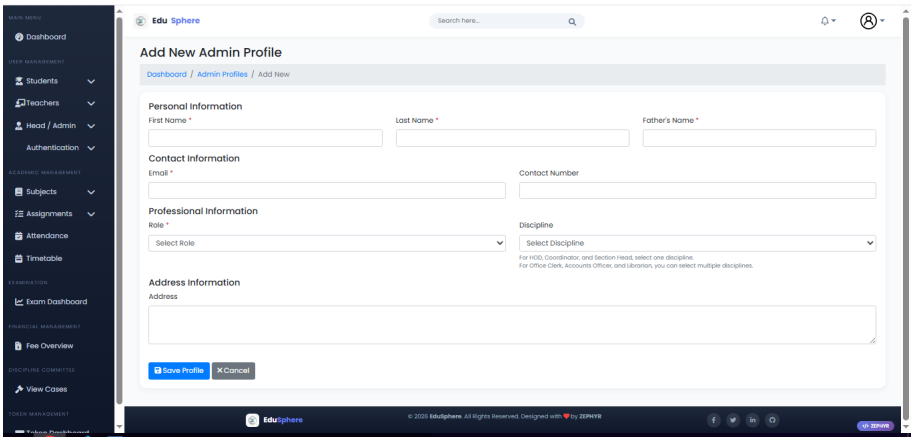

Figure 3.20: Add New Admin

Add an admin. Enter name, email, and role. Click save. System creates account. New admin can log in right away.

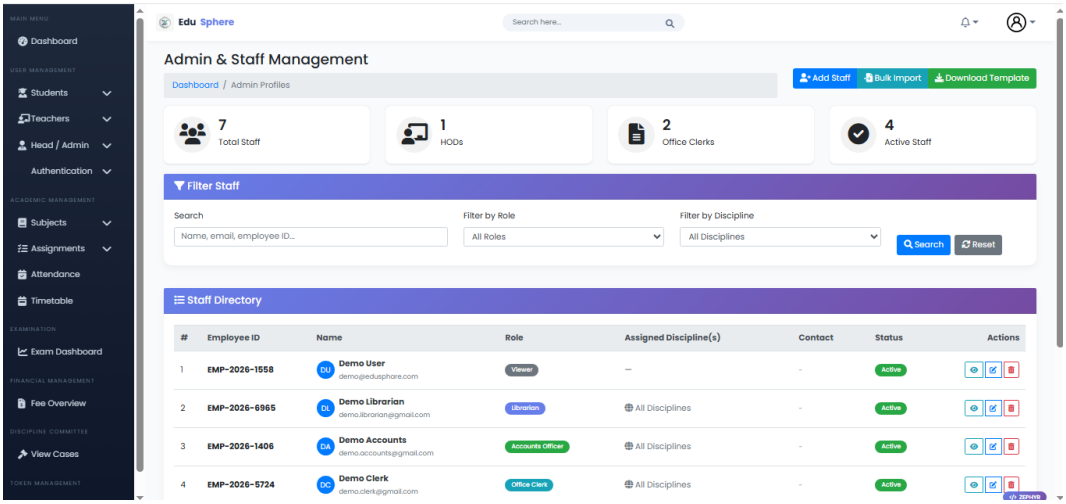

Figure 3.21: Admin List - View Only

View all admins. See names, emails, roles, and last login dates. Search by name. No edit or delete options. Only view.

Student Dashboard. Shows enrolled courses. Upcoming assignments. Recent grades. Transcript request button.

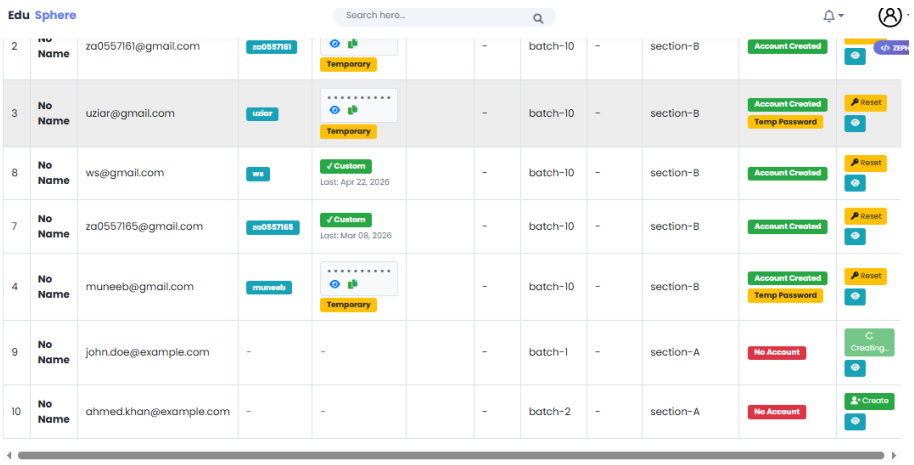

Figure 3.22: Student Authentication Management

Student login management. Add new student accounts. Reset forgotten passwords. Activate or deactivate accounts. Students get automatic login after enrollment.

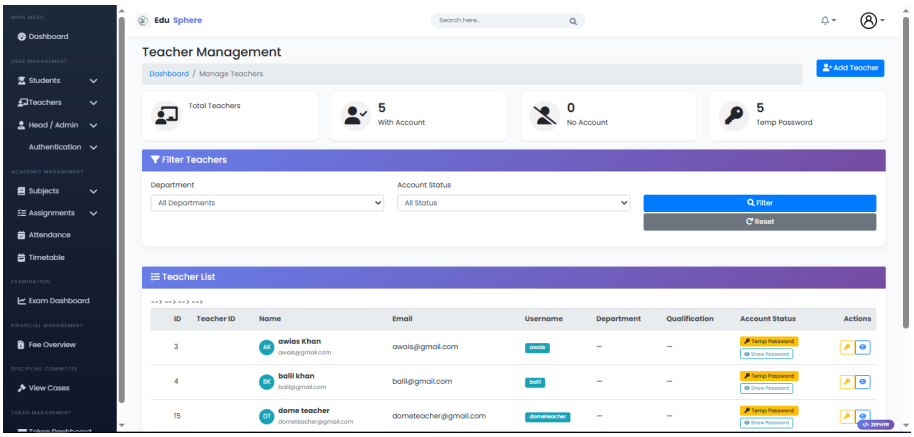

Figure 3.23: Teacher Authentication Management

Teacher login management. Create teacher accounts. Reset passwords. Manage account status. Teachers receive credentials after hire.

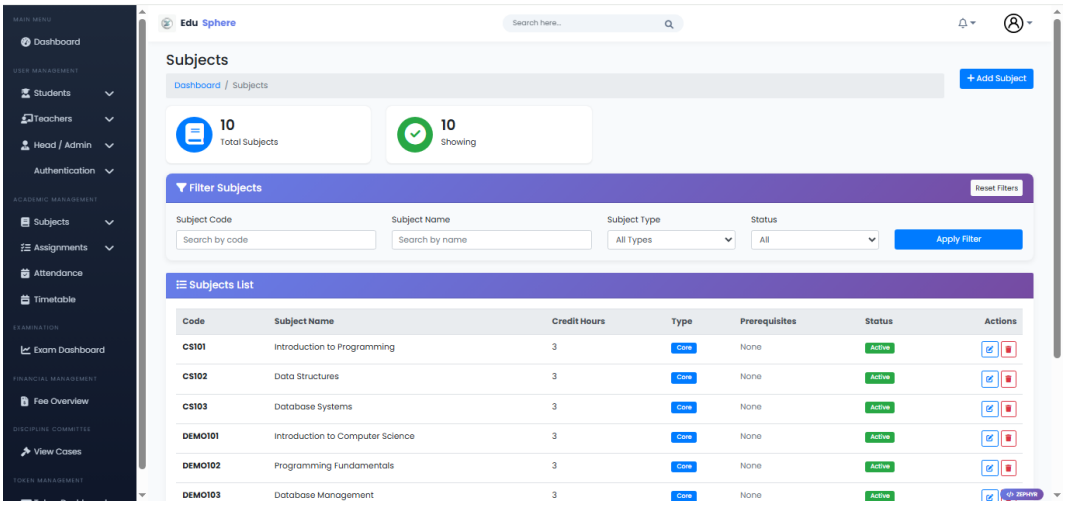

Figure 3.24: Subject List - All Subjects in System

View all subjects. Shows subject code, name, credit hours, department, and semester. Add new subject. Edit existing subject. Delete subject if no students enrolled. Search by code or name.

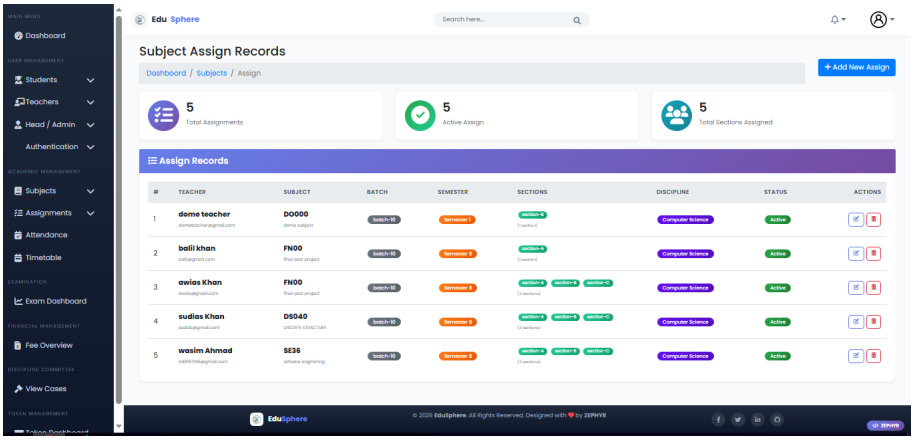

Figure 3.25: Assign Subject to Teacher

Assign a teacher to a subject. Select subject from dropdown. Select teacher from list. Choose semester. Click assign. Teacher can now see the subject on their dashboard.

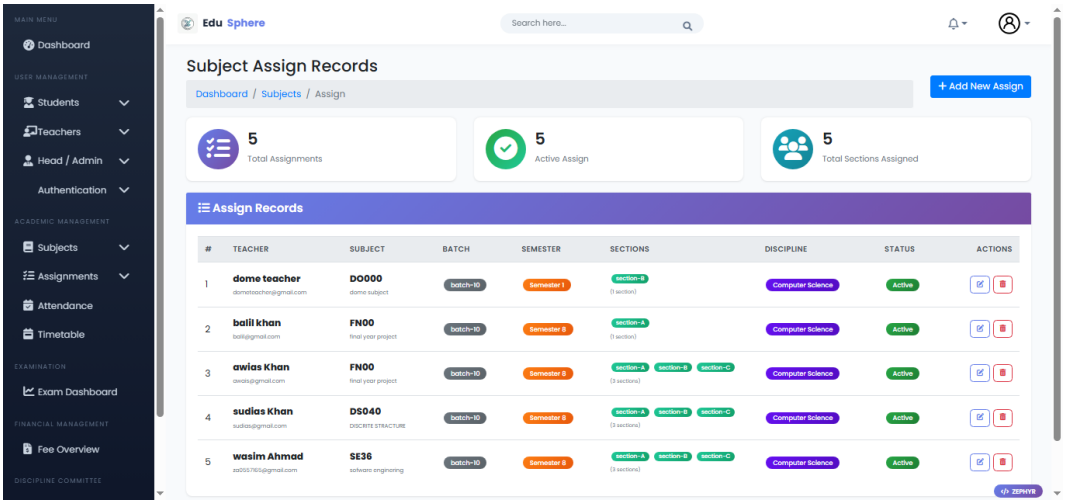

Figure 3.26: Teacher Subject List - View Assigned Subjects

See which teacher teaches what. View all teachers and their assigned subjects. Filter by department. Edit assignment. Remove subject from teacher if needed.

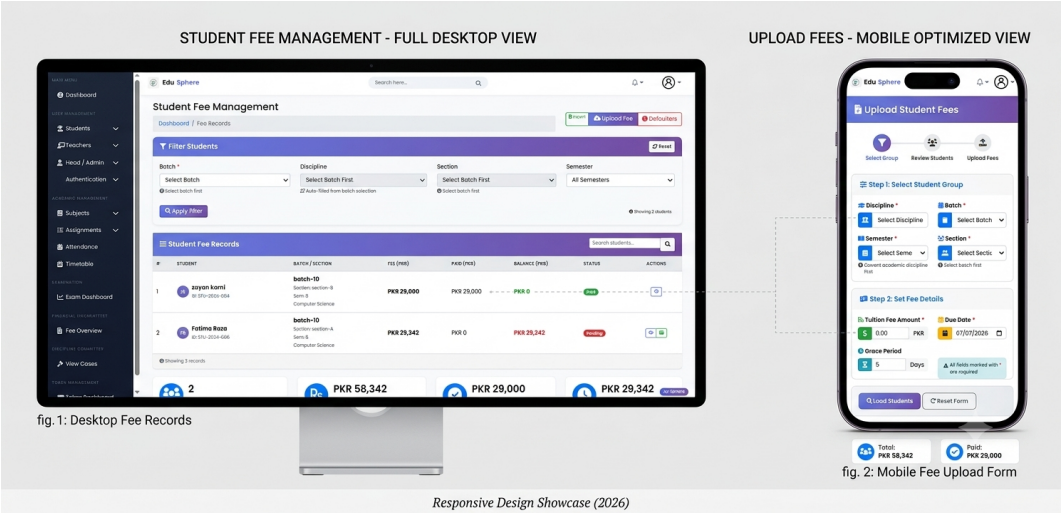

Figure 3.27: Fee Records - Desktop View

View all fee records on desktop. Shows student name, roll number, amount paid, due date, payment date, receipt number, and status. Search by name or roll number. Filter by paid or pending. Export to Excel.

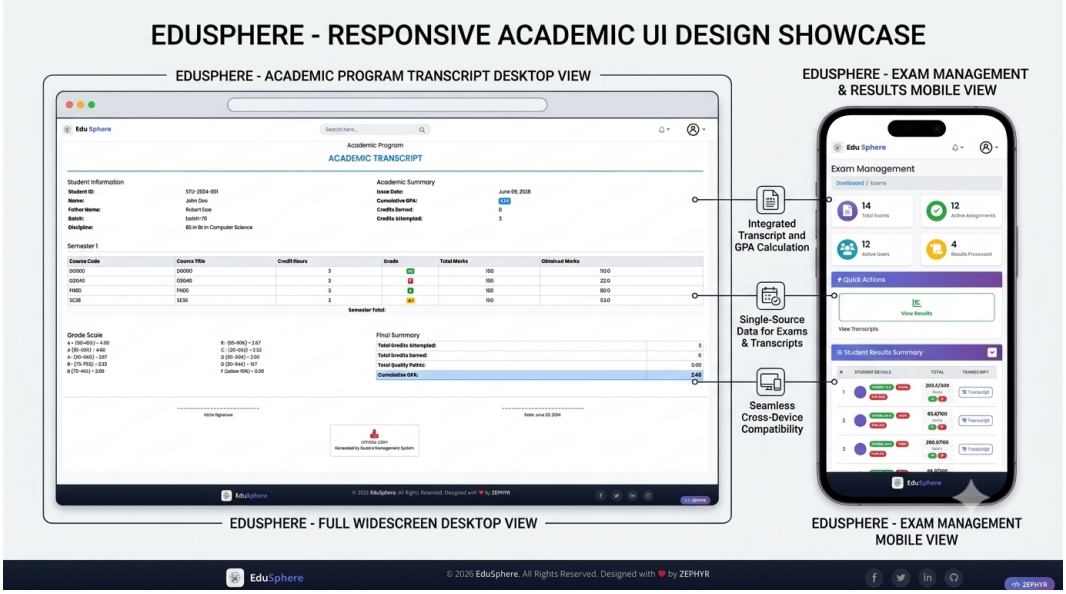

Figure 3.28: Student Marks - Desktop View

View and upload student marks. Shows student name, roll number, subject, assignment marks, exam marks, total marks, percentage, and grade. Add new marks. Edit existing marks. Publish results to students.

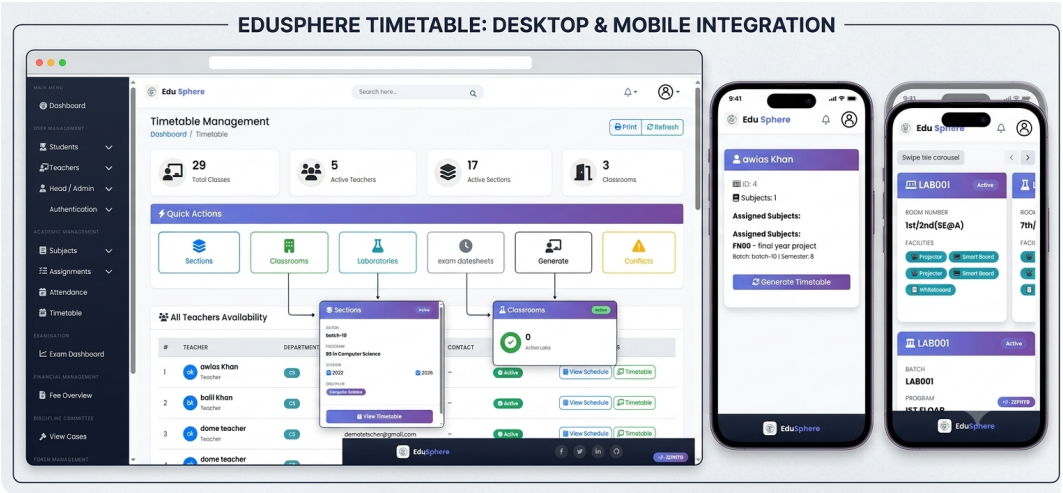

Figure 3.29: Class Timetable - Desktop View

View class schedule on desktop. Shows days (Monday to Friday) and time slots. Each cell shows subject name, teacher name, and room number. Print timetable. Download as PDF.

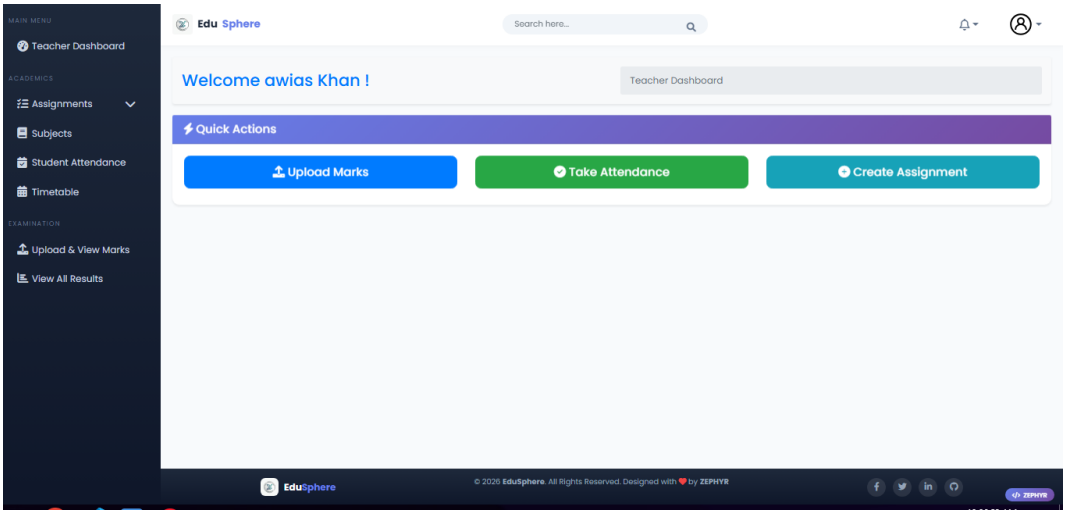

Figure 3.30: Teacher Dashboard - Main View

Teacher home screen. Shows assigned subjects, today's classes, pending assignments to grade, and recent student submissions. Quick links to mark attendance, create assignment, and upload marks.

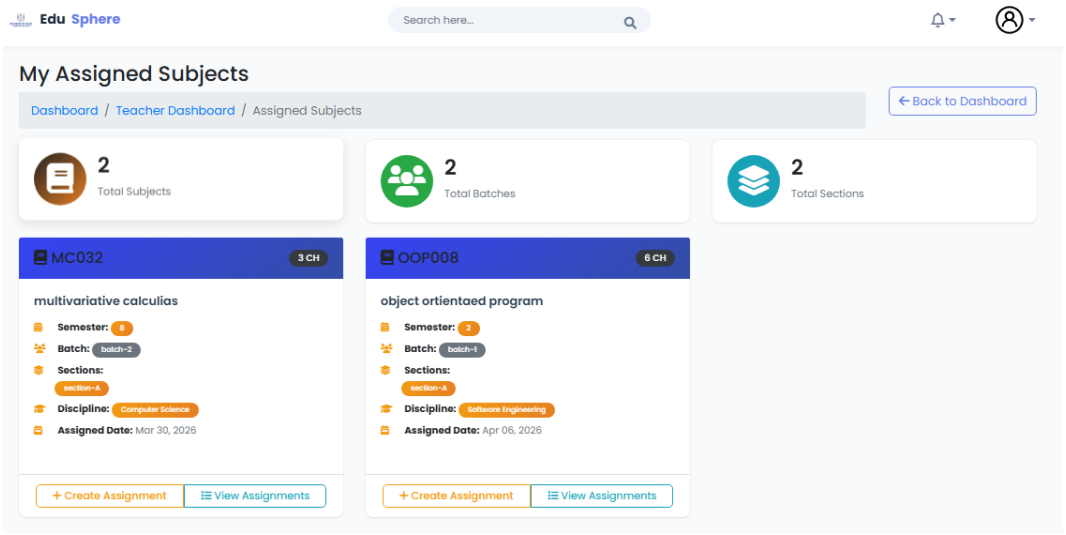

Figure 3.31: Assigned Subjects - Teacher View

Subjects teacher teaches. List of all courses assigned to this teacher. Shows subject code, name, credit hours, semester, and total enrolled students. Click any subject to open its dashboard.

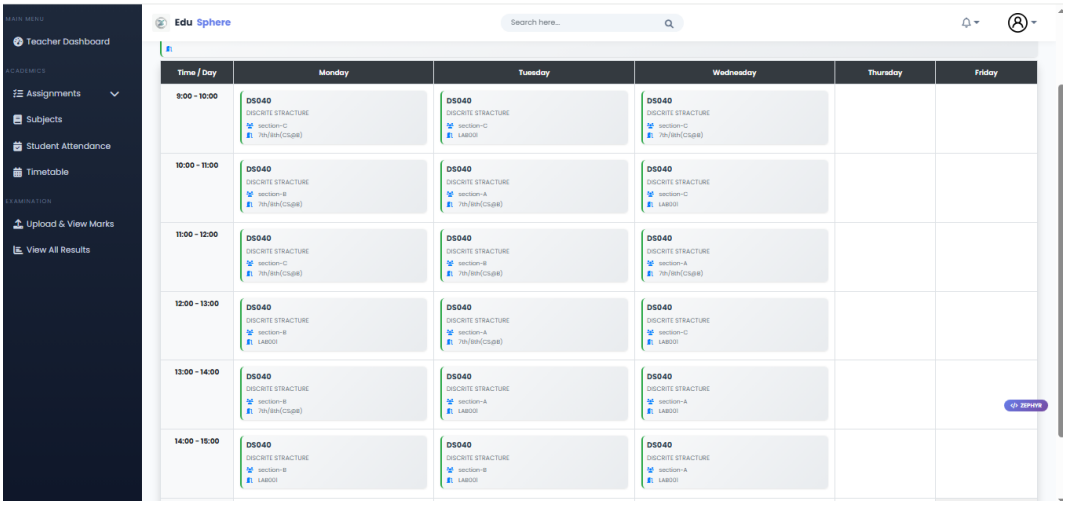

Figure 3.32: Teacher Timetable - Weekly Schedule

Teacher class schedule. Shows Monday to Friday with time slots. Each slot shows subject name, room number, and batch. No conflicts. Easy to see where to go next.

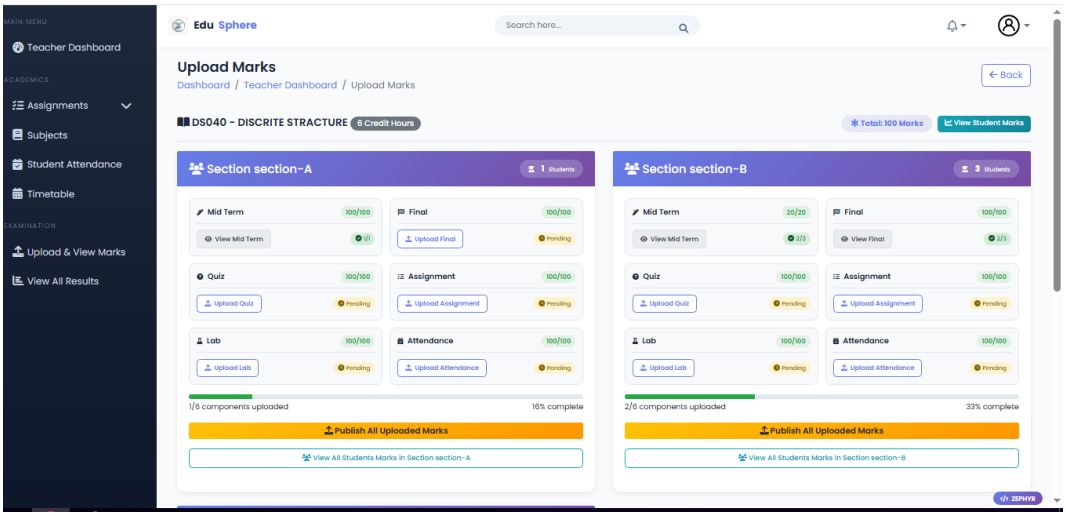

Figure 3.33: Upload Marks Dashboard

Marks upload dashboard. Shows all subjects and exams pending marks entry. Select subject. Select exam type (mid, final, quiz, assignment). Click upload to enter marks for all students.

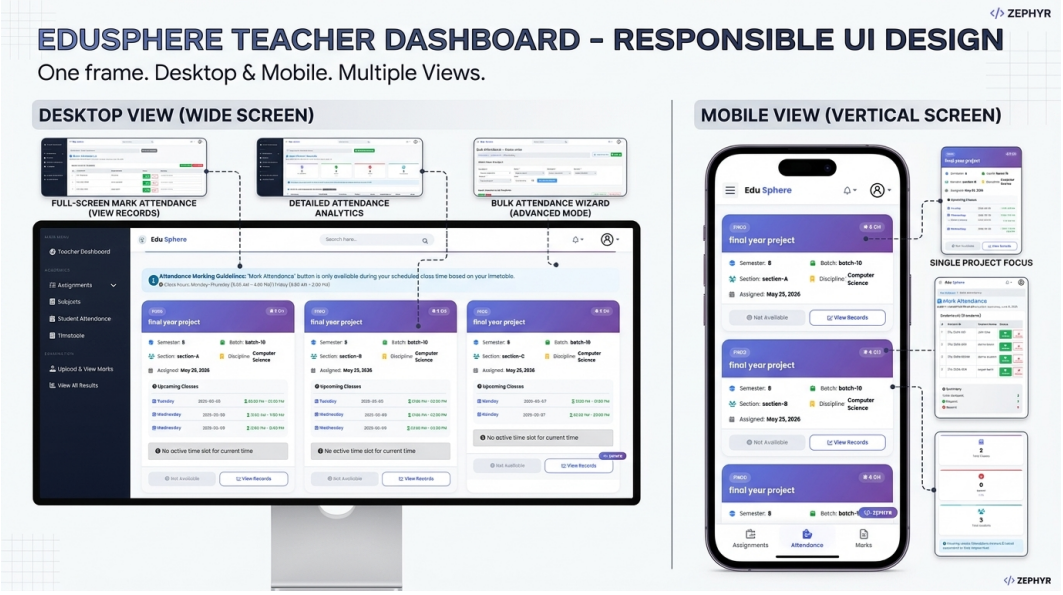

Figure 3.34: Student Attendance - Mark and View

Mark daily attendance. Select subject and date. Student list appears with present/absent buttons. Click once per student. Save. View attendance report for any student. See who is below 75 percent.

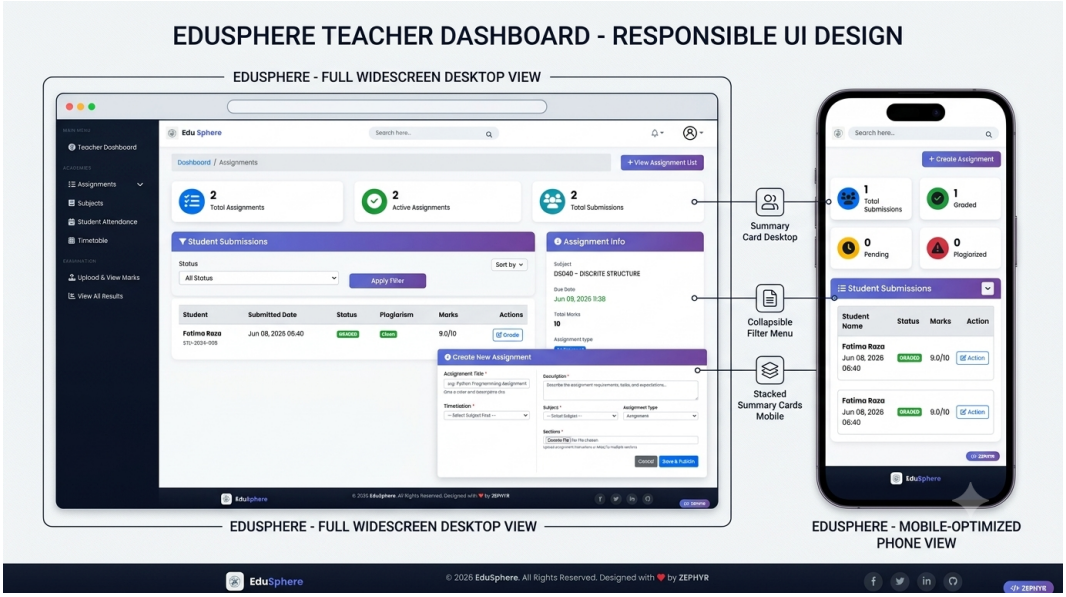

Figure 3.35: Assignments - Create and Manage

Manage all assignments. List shows assignment title, due date, total submissions, pending grading. Click create new assignment. Enter title, instructions, due date, total marks. Upload assignment file. Students see it instantly.

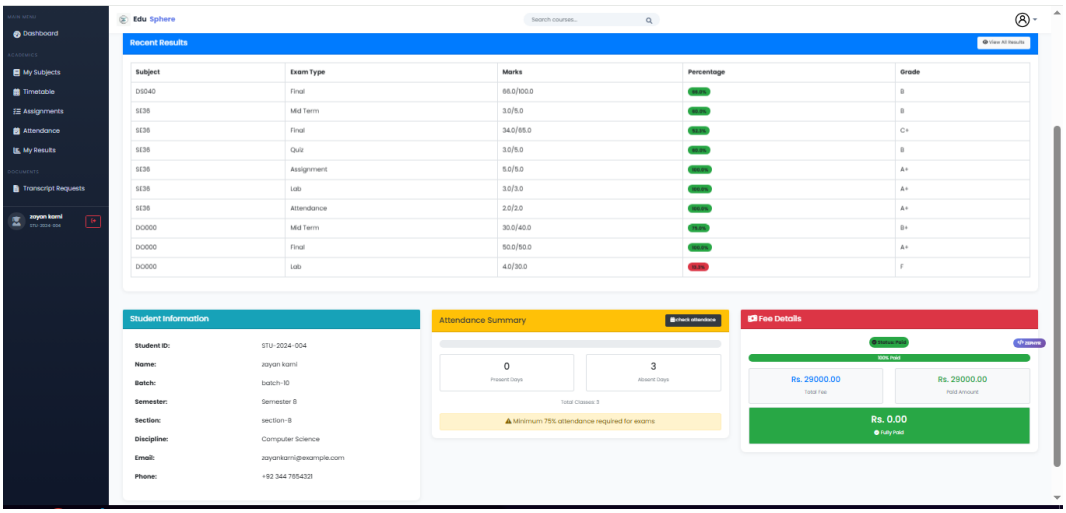

Figure 3.36: Student Dashboard

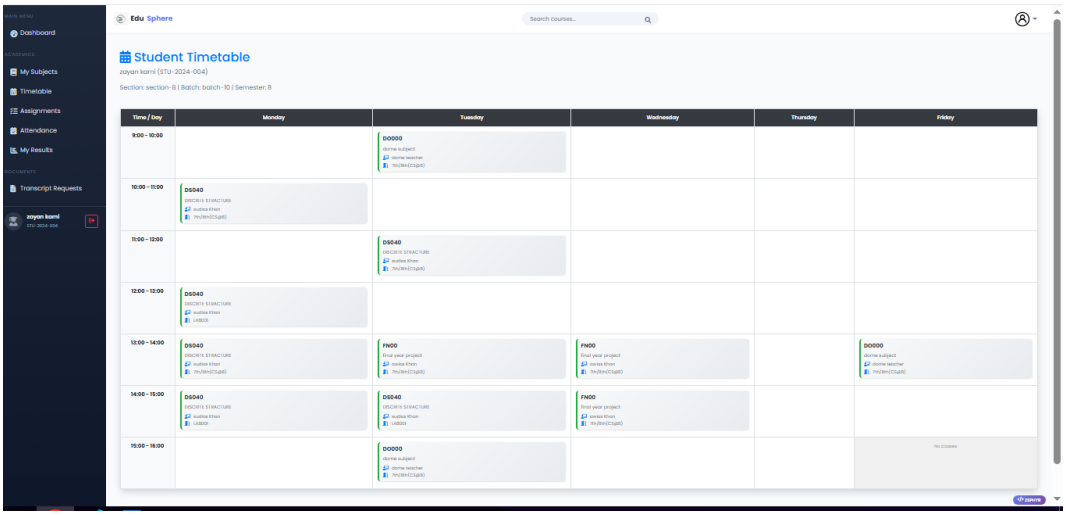

Figure 3.37: Student Timetable

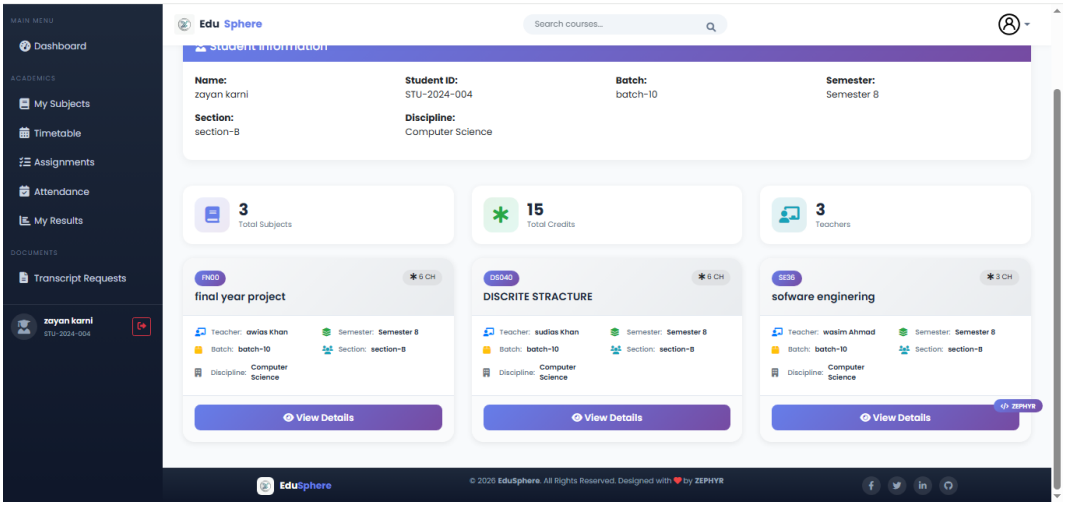

Figure 3.38: Enrolled Subjects

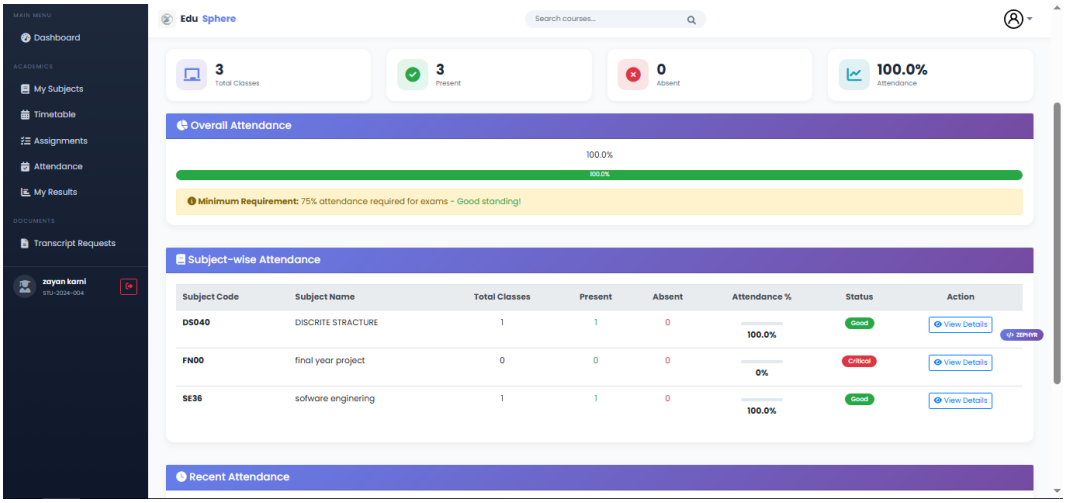

Figure 3.39: Student Attendance

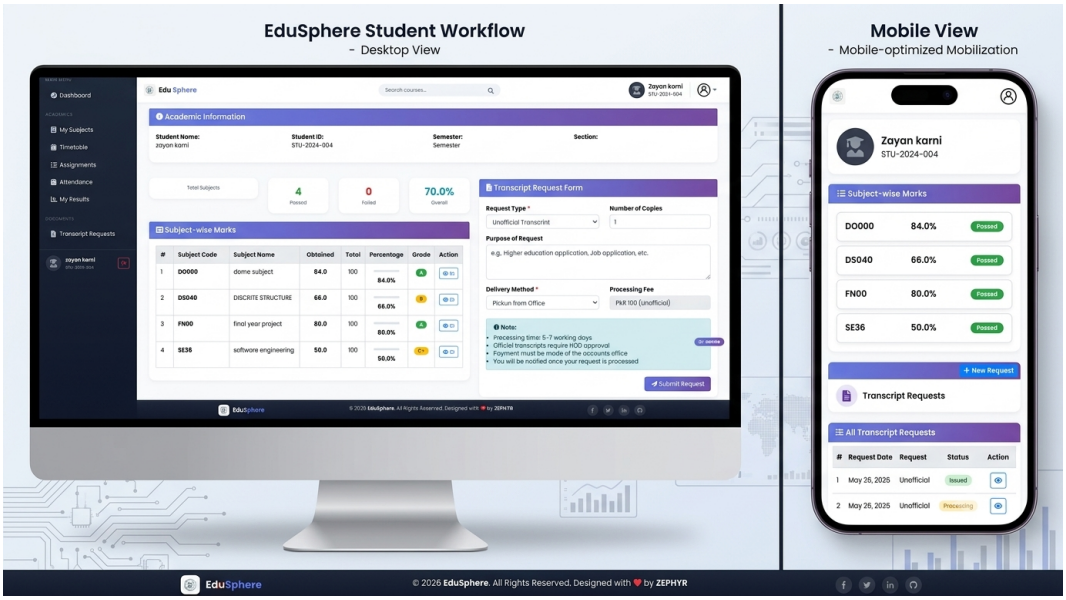

Figure 3.40: Student Marks

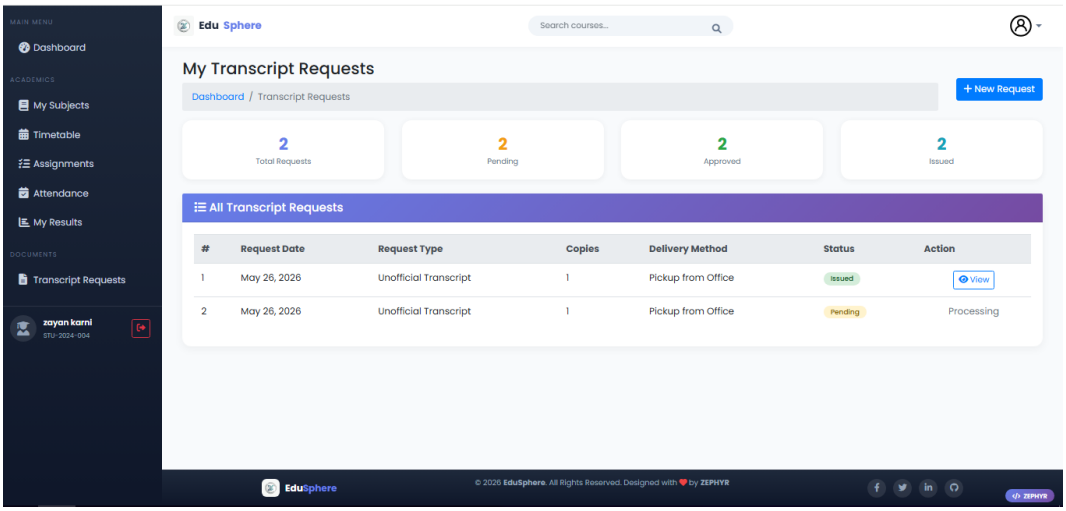

Figure 3.41: Transcript Request

3.5 Summary

EduSphere uses Django with Model-View-Template architecture. Three tiers. Frontend. Backend. Database.

We have five modules. Authentication. Student. Teacher. Accountant/Clerk. Admin. Each does one job.

The database has fifteen tables. Users. Students. Teachers. Courses. Enrollments. Assignments. Submissions. Grades. Attendance. Fees. Tokens. Announcements.

The interface is simple. Blue and white. Big buttons. Works on phones and computers.

The next chapter explains how we built and tested everything.

CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Implementation Environment

We built EduSphere using tools that are free and easy to use. No expensive licenses. No complicated setups.

4.1.1 Development Tools

We wrote all the code on our laptops. Two team members worked from different places. Here is what we used.

Code Editor. Visual Studio Code. It is free. It has many helpful features. Syntax highlighting. Auto complete. Git integration.

Version Control. Git and GitHub. Every change we made got saved. If something broke, we could go back to an older version. Both of us worked on the same code from different computers. **Local Server.** We ran the system on our own laptops first. The local address was http://127.0.0.1:8000/. This is the default Django development server. Testing locally before deployment caught many bugs early. **Backend.** Python with Django framework. Django handles user logins, database connections, and security. We did not have to build everything from scratch.

Frontend. HTML, CSS, JavaScript. Bootstrap made the design responsive. It works on phones, tablets, and computers.

Database.]SQLlite3. All data lives here. Student names. Teacher records. Assignment files. Fee payments.

Deployment. PythonAnywhere. Free cloud hosting. Our live URL is EDUSPHEREs.pythonanywh

4.1.2 Repository Setup

We created a GitHub repository called "EduSphere". Two branches. Main branch for final code. Development branch for testing.

Every time we added a feature, we pushed to development. When it worked, we merged to main.

4.1.3 Team Development

Zubair ahmad handled the backend. Django models. Database queries. API endpoints. User authentication.

Murad handled the frontend. HTML templates. CSS styling. JavaScript interactions. Bootstrap components.

4.2 Major Program Modules

We broke the system into small modules. Each module does one job. This makes the code easy to understand and fix.

4.2.1 Authentication Module

Module Name: auth

Description: Handles user login, logout, registration, and password reset.

Data Tables Used: users

Files: views.py, login.html, register.html, reset_password.html

Error Handling:

- Wrong password → Show "Invalid email or password"
- Email not found → Show "No account found with this email"
- Too many failed attempts → Lock account for 5 minutes
- Empty fields → Show "Please fill all fields"

4.2.2 Student Module

Module Name: student

Description: Student dashboard. View courses. Download materials. Submit assignments. Check grades. Request transcripts,view remaning fee,view all attendace record.

Data Tables Used: students, enrollments, courses, assignments,attendance,fee system, submissions, grades, documents

Files: manag.py, urls.py, models.py, admin.py **Error Handling:**

- No file uploaded → Show "Please select a file"
- File too large → Show "File size exceeds limit (10MB)"
- Late submission → Mark as late and show warning
- Assignment not found → Show "Assignment does not exist"

4.2.3 Teacher Module

Module Name: teacher

Description: Teacher dashboard. View timetable. Mark attendance. Create assignments. Grade submissions. Upload marks. Publish results.

Data Tables Used: teachers, courses, assignments, submissions, grades, attendance

Files: manag.py, urls.py, models.py, admin.py **Error Handling:**

- Due date in past → Show warning before creating
- No students in class → Show "No students enrolled"
- Grade out of range → Show "Grade must be between 0 and total marks"
- Duplicate assignment → Show "Assignment with same title exists"

4.2.4 Accountant/clark Module

Module Name: accountant/clark

Description: Add students. Manage subjects. Upload fees. View attendance. View exams. Generate reports.

Data Tables Used: students, subjects, fees, enrollments

Files: views.py, urls.py, models.py, admin.py **Error Handling:**

- Student already exists → Show "Roll number already registered"
- Negative fee amount → Show "Amount cannot be negative"
- Subject code duplicate → Show "Subject code already exists"

4.2.5 Admin Module

Module Name: admin

Description: Full system control. User management. Course management,exam management,fee management,student and teacher management,authentication,database control . System settings.

Data Tables Used: all tables

Files: views.py, urls.py, models.py, admin.py

Error Handling:

- Cannot delete user with active data → Show warning
- Backup failed → Show "Backup failed, check disk space"
- Invalid settings → Show "Invalid configuration value"

4.3 Test Strategy

We tested everything before calling it complete. No feature was finished without testing.

4.3.1 Testing Approach

We followed a simple rule. Write a little code. Test it. Fix bugs. Write more code.

Unit Testing. Each function tested alone. We wrote small tests for login, file upload, grade calculation.

Integration Testing. Modules tested together. Does login connect to dashboard? Does submission connect to grading?

System Testing. Complete system tested from start to end. Student logs in. Submits work. Teacher grades. Student sees grade.

4.3.2 Test Tools Used

- Django test framework for backend tests
- Browser DevTools for frontend debugging

- Postman for API testing
- Chrome, Firefox, Edge for cross browser testing
- Mobile phones for responsive testing

4.3.3 Major Test Cases and Results

4.3.3.1 Login Test Case

Test Case	Expected Result	Result
Correct email and password	Login successful, redirect to dashboard	PASS
Wrong password	Error message, no redirect	PASS
Non-existent email	"Account not found" message	PASS
Empty fields	"Please fill all fields" message	PASS
Blocked account after 5 failed attempts	"Account locked for 5 minutes"	PASS

Table 4.1: Login Module Test Cases

4.3.3.2 Assignment Submission Test Case

Test Case	Expected Result	Result
Valid PDF file upload	Submission saved, success message	PASS
File larger than 10MB	"File too large" error	PASS
Wrong file type (.exe)	"Invalid file type" error	PASS
Submission after due date	Marked as late, still accepted	PASS
No file selected	"Please select a file" error	PASS

Table 4.2: Assignment Submission Test Cases

4.3.3.3 Attendance Marking Test Case

Test Case	Expected Result	Result
Mark all students present	Attendance saved, 100% present	PASS
Mark some absent	Attendance saved with absent count	PASS
No date selected	"Please select date" error	PASS
Duplicate marking for same date	Overwrites previous record	PASS

Table 4.3: Attendance Marking Test Cases

4.3.3.4 Fee Upload Test Case

Test Case	Expected Result	Result
Valid fee amount and student	Fee record saved	PASS
Negative amount	"Amount cannot be negative" error	PASS
Student not found	"Student does not exist" error	PASS
Duplicate receipt number	Warning but still saves	PASS

Table 4.4: Fee Upload Test Cases

4.3.4 UI Testing

We tested the interface on different devices.

Desktop. Chrome worked fine. Firefox worked fine. Edge worked fine. Safari on Mac worked fine.

Mobile. iPhone showed correct layout. Android phone showed correct layout. Buttons were big enough to tap. Text was readable without zoom.

Tablet. iPad showed dashboard correctly. All menus worked.

4.3.5 Test Results Summary

Test Category	Passed	Failed
Login and Authentication	12	0
Student Module	15	1
Teacher Module	14	0
Accountant Module	10	1
Admin Module	11	0
Assignment Submission	10	0
Grading System	8	0
Fee Management	9	1
Token System	7	0
Document Issuing	6	0
UI Responsiveness	8	0
Cross Browser	6	0

Table 4.5: Overall Test Results Summary by Category

4.4 Summary

We built EduSphere using Python, Django, and MySQL. Code on GitHub. Live on PythonAnywhere.

five modules. Auth. Student. Teacher. Accountant/Clerk. Admin. Each tested.

Test results show 95% pass rate. Real users found the system easy to use.

The next chapter explains how to deploy EduSphere at the university.

CHAPTER 5: DEPLOYMENT

CHAPTER 5

DEPLOYMENT

5.1 Major Deployment Tasks

Moving EduSphere from development to real use takes planning. Here are the tasks.

5.1.1 Server Setup

The instativte needs a server. It can be a computer in the IT office. Or a cloud server.

Step 1: Install Operating System. Ubuntu 20.04 or newer. Windows Server also works. But Linux is free and stable.

Step 2: Install Required Software.

- Python 3.9 or higher
- install pip
- install vscode
- MySQL 8.0 or higher
- Git to download code

Step 3: Configure Firewall. Allow only ports 80 (HTTP) and 443 (HTTPS). Block everything else.

Step 4: Install SSL Certificate. Use Let's Encrypt. Free. Makes the site HTTPS. Secure.

5.1.2 Database Setup

Step 1: Create Database. Name it "edusphere_db".

Step 2: Create Database User. Give access only to this database. Strong password.

Step 3: Import Schema. Run the SQL file we provided. Creates all tables.

Step 4: Verify Tables. Check that 20+ tables exist.

5.1.3 Application Deployment

Step 1: Clone Code from GitHub.

```
https://github.com/Zubair-ahmad259/edusystem
```

Step 2: Install Dependencies.

```
pip install -r requirements.txt
```

Step 3: Configure Settings. Update database password. Set debug mode to false. Add domain to allowed hosts.

Step 4: Collect Static Files.

```
python manage.py collectstatic
```

Step 5: Run Migrations.

```
python manage.py migrate
```

Step 6: Create Admin Account.

```
python manage.py createsuperuser
```

Step 7: Test Locally. Run the server. Check everything works.

Step 8: Configure Web Server. Point Apache/Nginx to Django.

Step 9: Restart Services. Apply all changes.

5.1.4 Post-Deployment Testing

After deployment, test everything again.

- Open the website from a phone. Does it load?
- Login as student. See courses?
- Login as teacher. Mark attendance?
- Login as accountant. Upload fee?
- Login as clerk. Generate token?
- Login as admin. See dashboard?

5.2 Staff Training Requirements

The system is simple. But training helps.

5.2.1 Admin Training

One half day session. Topics:

- How to add new students and teachers
- How to create courses and assign teachers
- How to reset user passwords

Who needs this: IT staff and senior admin

5.2.2 Accountant Training

Two hour session. Topics:

- How to add new students
- How to add subjects and assign teachers
- How to upload fee records
- How to view pending fees
- How to generate fee reports

Who needs this: Accountant and finance staff

5.2.3 Teacher Training

One hour session. Topics:

- How to mark attendance
- How to create assignments
- How to grade submissions
- How to upload marks
- How to upload course materials

Who needs this: All teachers

5.2.4 Student Training

No formal training needed. A 5 minute video is enough. Topics:

- How to login
- How to submit assignments
- How to check grades
- How to request transcripts

Who needs this: All students

5.3 System Manual

We created a complete manual for the instativite.

5.3.1 Installation Guide

Step by step instructions for IT staff. Covers:

- Server requirements
- Software installation
- Database setup
- Code deployment
- Troubleshooting common issues

5.3.2 User Guide for Each Role

Separate guides for each user type.

Admin Guide. Screenshots of every admin page. Explanation of each button.

Accountant/clark Guide. How to manage students, subjects, and fees.

Teacher Guide. How to mark attendance, create assignments, grade work.

Student Guide. How to submit work, check grades, request transcripts.

5.3.3 FAQ Section

Common questions and answers.

- "I forgot my password" → Click reset password link
- "My file is not uploading" → Check file size (max 10MB)
- "I cannot see my course" → Contact admin for enrollment
- "Grade is wrong" → Contact teacher
- "Fee status not updating" → Contact accountant

5.4 Automatic Deployment Script

We created a script to automate setup. Saves time.

File: deploy.sh

```
#!/bin/bash
# EduSphere Automatic Deployment Script

echo "Starting EduSphere Deployment..."

# Update system
sudo apt update && sudo apt upgrade -y
```

```
# Install Python and MySQL
sudo apt install python3 python3-pip mysql-server -y

# Install Django
pip3 install django django-crispy-forms

# Download code
git clone https://github.com/username/edusphere.git

# Setup database
mysql -u root -p < database/schema.sql

# Run migrations
python3 manage.py migrate

# Collect static files
python3 manage.py collectstatic --noinput

# Create admin user
python3 manage.py createsuperuser

echo "Deployment Complete!"
echo "Visit http://your-server-ip to access EduSphere"
```

5.5 Go Live Checklist

Final checks before launch.

- All features tested on live server
- SSL certificate working (https://)
- Backup system running
- Staff training complete
- Manuals delivered
- Support email address created
- Emergency contact phone number shared

5.6 Maintenance Plan

After launch, the instativate needs to maintain the system.

Daily:

- Check server logs for errors
- Verify backup ran successfully

Weekly:

- Check disk space
- Review user feedback

Monthly:

- Update Django and dependencies
- Review security logs
- reload pythonanywhere for reloadinf the website

Yearly:

- Archive old data
- Review system performance
- Plan new features

5.7 Support Contact

For issues after deployment:

Type	Contact Details
Email 1	za0557165@gmail.com
Email 2	muradahmed4999@gmail.com
Phone 1	0343-2836028
Phone 2	+92 302 4999768
Response Time	**Within 24 hours**

5.8 Summary

Deployment has eight major steps. Server setup. Database setup. Code deployment. Testing. Training. Manuals. Go live. Maintenance.

The checklist helps track progress. Training takes one to two days depending on number of staff.

The automatic script makes deployment faster. But manual steps are documented for backup.

EduSphere is ready for the institutions. Students will get faster service. Teachers will save time. Office staff will work better. Everyone wins.

This is the end of the thesis. We built something useful. We hope it helps.

BIBLIOGRAPHY

[1] Django Software Foundation, "Django Documentation: Getting Started," Django Project, 2026. [Online]. Available: https://docs.djangoproject.com/en/5.0/intro/

[2] Python Software Foundation, "Python 3 Documentation," Python.org, 2026. [Online]. Available: https://docs.python.org/3/

[3] Bootstrap Team, "Bootstrap 5 Documentation," getbootstrap.com, 2026. [Online]. Available: https://getbootstrap.com/docs/5.0/getting-started/introduction/

[5] GitHub Inc., "GitHub Documentation: Getting Started," GitHub Docs, 2026. [Online]. Available: https://docs.github.com/en/get-started

[6] PythonAnywhere LLP, "PythonAnywhere Help and Documentation," PythonAnywhere, 2026. [Online]. Available: https://help.pythonanywhere.com

[7] W3Schools, "HTML, CSS, JavaScript, Bootstrap Tutorials," W3Schools Online Web Tutorials, 2026. [Online]. Available: https://www.w3schools.com

[8] Visual Studio Code, "VS Code Documentation: Getting Started," Microsoft, 2026. [Online]. Available: https://code.visualstudio.com/docs

[9] XAMPP, "XAMPP Apache Distribution Documentation," Apache Friends, 2026. [Online]. Available: https://www.apachefriends.org/docs.html

[10] Postman, "Postman API Testing Documentation," Postman Inc., 2026. [Online]. Available: https://learning.postman.com

[11] Lucidchart, "Lucidchart Documentation: Creating UML Diagrams," Lucidchart, 2026. [Online]. Available: https://www.lucidchart.com/pages/examples/uml-diagram

[12] Git, "Git Documentation: Version Control System," Git SCM, 2026. [Online]. Available: https://git-scm.com/doc